

chapter_3_notes.pdf
PDF

我在复习计算机网络的课程。附件是第3章的slides,, 请你按照课件的思路, 详细的给我过。不要一下子输出所有, 而是输出一节解释, 让我理解一下, 并给我提问的机会。如果我没问题了再继续

已思考 7s >

好, 我们按课件的顺序来, 从 **3.1 Transport-layer services (传输层服务)** 这一节开始 (对应你这份 slides 的前几页)。  chapter_3_notes

3.1 传输层到底在“提供什么服务”?

1) 传输层的核心作用: 给应用“看起来像直接连着一样”的通信

课件一句话是: **transport layer 提供运行在不同主机上的“应用进程之间”的逻辑通信 (logical communication)**。  chapter_3_notes

- “逻辑通信”= 让应用感觉自己在和对方应用直接对话
- 但实际上中间经过: 网络层/链路层/物理层、路由器、各种链路.....

对比一下 (这是本节最容易考的区分点):

- **网络层 (IP):** 提供 *host-to-host* (主机到主机) 的逻辑通信
- **传输层 (TCP/UDP):** 提供 *process-to-process* (进程到进程) 的逻辑通信 (靠端口/套接字来做到)  chapter_3_notes

课件的“房子/孩子/信件”类比就是为了强调这一点: 房子=主机, 孩子=进程, 邮政=网络层, 家里分信的人=传输层 demux。  chapter_3_notes

2) 传输层在端系统做什么 (发送端/接收端的动作)

课件给的工作流是:  chapter_3_notes

发送端:

- 应用产生 message (应用层数据)
- 传输层把 message 切成 **segments (段)**

- 给每段加上传输层首部 (TCP/UDP header)
- 交给网络层 (IP) 去发

接收端：

- 从网络层拿到一个个 segment
- 传输层把段重组/交付给正确的应用进程 (通过 socket)
- 把数据交给应用层

这也对应课件里“Packet at different layers”的术语变化：message → segment → datagram → frame (按层不同叫法不同)。  chapter_3_notes

3) Internet 里两种传输层协议：TCP vs UDP (这节只要先有直觉)

课件这里先不讲细节，只给你“服务差异”的大框架：  chapter_3_notes

- **TCP：可靠、有序、面向连接**
 - 可靠交付 + 按序
 - 还有：流量控制、拥塞控制、连接建立/拆除
- **UDP：不可靠、无序、无连接**
 - “best effort”的轻量扩展
 - 没有连接建立 (更省延迟/状态)
 - 没有拥塞控制 (可“猛发”，但也因此可能更不友好)

课件还强调：一些服务在 Internet 传输层里就是“不提供”的，例如“带宽保证”“时延保证”。  chapter_3_notes

我确认你理解的小问题 (你回答完我再继续 3.2)

1. 用一句话区分：网络层 vs 传输层 的通信对象分别是什么？
2. 发送端传输层做的两件关键事是什么？ (用课件原话也行：一个是“切”，一个是“加”)
3. 你现在对“logical communication”这句话的直觉是什么：它在“骗”应用什么？

来源



1. 网络层传输层分成一个个segment并带好传输层头的
应用层数据2. 将应用层数据切成segments，加上传输层头

3. 骗应用之间一直在直连通话（但是我不太理解，难道不是每层都可以这样看吗）

你答得很接近课件的点了，我帮你把关键概念“对齐”一下，然后专门把你困惑的第 3 点讲透。

先纠正/精炼一下你的 1)

你写的那句更像是在描述“传输层做了什么”，但题目要你区分“网络层 vs 传输层”的通信对象：

- **网络层 (IP)**：提供 **主机到主机** (host-to-host) 的逻辑通信。它只关心把 datagram 从主机 A 送到主机 B (地址是 IP)。
- **传输层 (TCP/UDP)**：提供 **进程到进程** (process-to-process) 的逻辑通信。它关心把数据交给 **主机 B 上的正确应用进程** (靠端口/套接字)，并且 TCP 还提供可靠/有序等附加服务。

你第 2 点 “切 segment + 加传输层头” 完全对。

你问得很关键：logical communication 难道不是每层都能这么看吗？

是的！几乎每一层都可以说自己在提供“逻辑通信”。这不是矛盾，而是“分层抽象”的本质：

每一层都“假装”自己和对端同层在直接通信，但其实是靠下层实现的。

区别在于：每一层假装“直连”的对象不一样、提供的服务也不一样。

用同一个真实场景，把三层的“骗法”拆开

你在浏览器访问网页：

1. 应用层的逻辑通信

浏览器“感觉”自己在和对方的 Web 服务器应用 (HTTP) 对话。

它不关心分段、丢包、路由、MAC 地址。

2. 传输层的逻辑通信（你当前这章重点）

浏览器进程“感觉”自己在和对方 **某个服务器进程的某个端口** 直接对话（比如 TCP 连接到 443）。

它关心的是：这条“进程到进程的管道”能不能可靠、有序、控速等 (TCP) ——或者就尽快发出去 (UDP)。

3. 网络层的逻辑通信

主机 A “感觉”自己在把 IP datagram 直接送到主机 B (目的 IP)。

它不管主机 B 上到底哪个进程接收——那是传输层端口的事。

一句话记忆

- 每层都在“骗”你有一条直连通道
- 但骗的粒度不同：网络层骗“主机直连”，传输层骗“进程直连”，应用层骗“应用语义直连”。

我再用一个小问题确认你真的通了

如果一台服务器（同一个 IP）上同时跑着 Web(443) 和 SSH(22)，IP 层把包送到这台主机后，谁负责把数据交给 443 的进程还是 22 的进程？靠的是什么信息？

你答完如果没问题，我们就进入下一节 3.2 Multiplexing & Demultiplexing（多路复用/分用），它正好就是回答这个问题的。



1. 传输层负责具体到进程的对话，利用传输层头中的信息

对，完全对齐课件思路了。接下来我们进入 3.2 Multiplexing & Demultiplexing（多路复用/分用）：也就是“同一台主机上这么多进程，传输层怎么把数据送对人”。

3.2 多路复用与分用在干嘛

1) 你刚才那句话的标准版

- 分用（demultiplexing）：接收端把“到达主机的 segment”交给正确的 socket/进程
- 多路复用（multiplexing）：发送端从多个 socket/进程收集数据，封装成 segment 交给网络层发出去

核心机制：socket + 端口号（port numbers）（端口号在 TCP/UDP 首部里）。

2) 分用怎么做：看首部字段“匹配哪个 socket”

接收端会用 segment 首部中的信息去匹配一个 socket（你刚才说的“传输层头中的信息”就是这句）。

UDP 的分用（简单）

UDP socket 通常主要按：

- **目的端口号**（以及本机 IP）来匹配

直觉：只要你发到这台主机的某个 UDP 端口，就会交给绑定该端口的那个 socket。

TCP 的分用（更精细）

TCP 因为“面向连接”，同一个服务器端口（比如 443）可能同时服务很多客户端连接，所以要用 **4 元组** 区分连接：

- 源 IP、源端口、目的 IP、目的端口

直觉：服务器上可以有很多条并行 TCP 连接都“目的端口=443”，但靠源 IP/源端口不同区分开，每条连接对应不同的 socket（或连接状态）。

3) 一个最容易考的场景：Web 服务器为什么能同时服务很多客户端？

因为：

- 服务器有一个监听 socket (listen) 绑定 443
- 每来一个客户端连接，内核根据 4 元组创建/分配一个新的连接 socket
- 后续每个到来的 TCP segment，都按 4 元组分到对应连接 socket

（你后面复习到 listen/accept 那一块，会发现它其实就是在操作这个“分用结果”。）

小测：确认你掌握 UDP vs TCP 分用差异

给你一个服务器 IP=1.2.3.4，端口 80。两个客户端同时访问：

- 客户端 A：10.0.0.1:50000 → 1.2.3.4:80
- 客户端 B：10.0.0.2:50001 → 1.2.3.4:80

问题：

1. 对 TCP 来说，这两条连接在服务器端会对应 **几个 socket**？为什么？
2. 如果是 UDP 且服务器只绑定了 80 端口，通常会对应 **几个 socket**？为什么？

你答完我们继续下一小节（会讲“无连接分用 vs 面向连接分用”的图和典型题）。



回答问题之前我先问：端口是有绝对的udp端口和tcp端口之分吗；你说的web服务器监听socket是welcomesocket吗，是既然这么说了就说明web服务用tcp吗

你这三个问题都问到点子上了，我按顺序回答：

1) 端口号是“绝对的 UDP 端口 / TCP 端口”吗？

端口号的数字本身（0-65535）是同一套编号，但在操作系统里是按“协议”分开的。

更精确地说：一个 socket 的绑定/匹配不是只看“端口号”，还看 **协议类型（TCP/UDP）**（以及本地 IP 等）。所以：

- **TCP 80** 和 **UDP 80** 可以在同一台机器上同时被不同进程占用，互不冲突。
- 但同协议里，比如两个进程都想绑定 **TCP 80**（同本地IP条件下），通常就会冲突（除非用一些特殊选项/场景，比如 `SO_REUSEPORT` 之类，课内一般不展开）。

你可以把它记成：端口号是“标签”，但真正唯一的是“协议 + 端口（+ 本地IP）”。

2) 你说的 web 服务器监听 socket 是 welcomeSocket 吗？

是的，课件/教材里常把服务器端的监听 socket 叫 **welcome socket**（或者 listening socket）。

- **welcome/listen socket**：只负责“迎接新连接请求”（被动打开，listen）
- **connection socket**：`accept()` 之后为某个具体客户端连接创建的 socket，后续收发数据用它

3) 这是否说明 Web 服务用 TCP？

在课程默认语境里，**是的：HTTP 基本是跑在 TCP 上**（经典的 HTTP/1.1、HTTP/2 都是 TCP）。所以你会看到：

- HTTP：TCP 80
- HTTPS：TCP 443

但补一句现实世界的扩展（不影响你做大多数题）：**HTTP/3 是跑在 QUIC 上，而 QUIC 用 UDP**，所以现代浏览器也可能用 UDP/443 来承载 HTTP/3。课程如果没专门讲 HTTP/3/QUIC，你就按“Web 用 TCP”来做题就对。

如果你这部分没疑问了，我们回到我刚才的小测题：

同一服务器 `1.2.3.4:80`，两个客户端 A、B 同时访问：

1. TCP 情况：服务器端对应几个 socket？为什么？
2. UDP 情况：如果只绑定 80 端口，通常几个 socket？为什么？



1. tcp下，对应1个welcome和两个socket一共3个；udp：1个？

已思考 33s >

你的答案基本满分 🍊 我把它用课件的“分用规则”再钉牢一下，然后我们就进入下一节 **3.3 UDP**。

先核对你刚才的两问

1) TCP：你说 “1 个 welcome + 2 个 connection = 3 个” ✅

对：服务器端会有一个监听/欢迎 socket (welcomeSocket / listening socket)，然后对每个客户端连接 accept() 出来一个 **connection socket**。而且 TCP 的分用是按 **4-tuple**（源IP、源端口、目的IP、目的端口）来区分，所以 A、B 两个客户端一定会落到两个不同的 connection socket。 chapter_3_notes

2) UDP：你说 “1 个” ✅（在“服务器只绑定一个端口”的前提下）

对：UDP socket 在课件里是按 **(dest IP, dest port)** 这个 **2-tuple** 来标识的；所以只要目的 IP/端口相同，即使来源 IP/端口不同，也会被分到 **同一个 socket/同一个进程**。

chapter_3_notes

（应用当然仍然能从 recvfrom 之类接口拿到“源IP/源端口”，用来回包；但那是应用逻辑，不是“分用到不同 socket”。）

3.3 Connectionless transport: UDP（无连接传输：UDP）

1) UDP 提供的服务是什么（先抓住一句话）

UDP 是“no frills / bare bones”的 Internet 传输协议，提供 best-effort：

- 可能丢包、可能乱序到应用
- **无连接**：发送方/接收方之间没有握手，每个 UDP segment 独立处理 chapter_3_notes

典型应用（课件举例）：流媒体（可容忍丢、对速率敏感）、DNS、SNMP。

chapter_3_notes

2) 为什么还要有 UDP (课件给的“存在理由”)

课件列的是这些现实动机：  chapter_3_notes

- 没有连接建立 (少了握手带来的延迟)
- 简单: 端系统不维护连接状态
- 首部小 (开销更低)
- 没有拥塞控制: 想多快发多快发 (“UDP can blast away as fast as desired”)

这句话很重要: 它既解释“为什么有人用 UDP”, 也暗示“为什么 UDP 可能更不友好/更容易把网络打爆” (后面拥塞控制章节会接上)。

3) UDP 报文段长什么样 (四个字段)

UDP 首部核心字段:

- source port
- dest port
- length (UDP 段总长度, 含首部)
- checksum  chapter_3_notes

4) UDP checksum 在干嘛 (你只要记住它的计算逻辑)

课件流程是:  chapter_3_notes

- 发送方把 (首部+数据) 当作一串 16-bit 整数做加法
- 做 one's complement (反码) 得到 checksum 填入字段
- 接收方重算 checksum 并比对: 不等则检测到错误; 相等也不保证“绝对没错” (可能存在漏检)

给你两个很短的自测题 (你答完我就继续 3.4 Reliable Data Transfer)

1. 用一句话说清楚: “UDP can blast away as fast as desired” 这句话指的是 UDP 缺少了哪个机制?  chapter_3_notes
2. UDP 的 socket 分用为什么说是 2-tuple (dest IP, dest port) 而不是 4-tuple?

 chapter_3_notes

来源



1. no congestion control 2. 只需目的地址和端口即可，头部小

已思考 19s >

很好，你两题都答对了  (UDP 就是“没有拥塞控制”，分用主要看目的端口/目的地址)。

我们进入 **3.4 Principles of Reliable Data Transfer (可靠数据传输原理)**，先讲这一节的“起步部分 + rdt1.0 + 为什么需要 rdt2.0”。不一次讲完。  chapter_3_notes

3.4 可靠传输：从“接口”开始 (Reliable data transfer: getting started)

课件先定义了一个抽象：我们要在一个 **不可靠信道 (unreliable channel)** 上实现可靠传输协议 rdt。它把 rdt 看成“夹在上层应用和下层不可靠信道之间的一层逻辑”。

 chapter_3_notes

你只要记住这四个“接口/事件”（后面所有 FSM 都围绕它们转）：  chapter_3_notes

- `rdt_send(data)`：上层（应用）把数据交给发送方 rdt
- `udt_send(packet)`：rdt 把 packet 丢给不可靠信道 (unreliable data transfer)
- `rdt_rcv(packet)`：接收方从信道收到 packet 时触发
- `deliver_data(data)`：接收方 rdt 把数据交付给上层应用

以及课件强调：我们先只考虑**单向数据传输**（data 单向），但控制信息（ACK 等）会双向流动。  chapter_3_notes

rdt1.0：假设信道完美可靠（“最简单基线”）

假设：不丢包、不比特错误。  chapter_3_notes

于是协议几乎啥也不用做：

- 发送方：`rdt_send(data) → make_pkt(data) → udt_send(pkt)`
- 接收方：`rdt_rcv(pkt) → extract(data) → deliver_data(data)`

 chapter_3_notes

这一步的意义：让你看到“FSM 的写法”和“协议就是围绕事件触发动作”。

为什么要升级到 rdt2.0：信道会“翻转比特”（bit errors）

下一步把信道变坏一点：packet 可能被损坏（bit error），但先不考虑丢包。

📄 chapter_3_notes

课件引入三个关键机制（这是后面所有版本的核心积木）： 📄 chapter_3_notes

1. 差错检测：checksum（让接收方能判断“这包坏了”）
2. 接收方反馈：ACK / NAK（告诉发送方“好/坏”）
3. 重传：发送方收到 NAK（或检测到问题）就重发

到这里，你可以把 rdt2.0 的直觉理解成“人类对话纠错”：对方没听清就让你再说一遍。课件也用这个类比提醒你。 📄 chapter_3_notes

我确认你跟上了没（2 个小问题）

1. 在 rdt 的抽象里，udt_send() 和 rdt_send() 的区别是什么？谁调用谁？
📄 chapter_3_notes
2. 从 rdt1.0 → rdt2.0，课件新增的机制可以概括成哪三类？（你用“checksum / feedback / retransmit”这种关键词就行） 📄 chapter_3_notes

你答完如果没问题，我们就继续下一小节：**rdt2.0 的致命 flaw（ACK/NAK 自己也可能坏）→ rdt2.1/rdt2.2 的序号与去 NAK 思路。**

来源

📄 🍌 🗨️ ⬆️ ↺️ ...

1. rdt层是应用层和传输层之间的“半层”，应用层数据先被 rdt的rdtsend方法分割成一个个packet，再由传输层继续传出去。用udtsend2. checksum, ack/nak, 重传。提问，当没有rdt机制的udp，这个类似分割应用层数据然后往下传的过程实际上是什么样的

已思考 31s >

你第 2 题完全对。第 1 题里有个小概念我帮你“拨正”一下：**rdt 不是“应用层和传输层之间的半层”，它在课件里是一个抽象的“可靠传输协议”，用来说明在不可靠信道上如何实现可**

靠交付；真实互联网里，这套思想主要体现在 **TCP 的可靠传输机制** 上。课件也明确说传输层协议运行在端系统、把 app message 切成 segments 交给网络层。  chapter_3_notes

你问的：没有 rdt 机制的 UDP，下传过程到底长什么样？

先把“rdt 在课件里的角色”对齐：课件这节先定义接口 `rdt_send` / `udt_send` / `rdt_rcv` / `deliver_data`，用来讨论可靠性要怎么补。  chapter_3_notes

而 **UDP 本身就是传输层协议**，它做的事更像是：

发送端 (UDP)

1. 应用调用 `sendto()` / `sendmsg()` 把一段数据交给 UDP socket
2. UDP 不会像 TCP 那样为了可靠性去“主动分段+重传+按序”；它通常把你这次 `sendto` 的数据当成一个 **UDP datagram**，加上 UDP 头（源端口/目的端口/length/checksum）  chapter_3_notes
3. 交给网络层 IP：封装成一个 IP datagram 发出去（再往下是链路层 frame）

课件里的那句“send side: breaks app messages into segments, passes to network layer”在 UDP 场景下也成立：**UDP 的“segment”往往就是你一次 send 的那坨数据 + UDP 头。**  chapter_3_notes

接收端 (UDP)

1. IP 把 datagram 交给 UDP
2. UDP 按 (**dest IP, dest port**) 这个 2-tuple 分用到对应 socket（所以不同来源也可能进同一个 socket）  chapter_3_notes
3. 校验 checksum（可检测比特错误），然后把 payload **原样交给应用**（可能丢、可能乱序——UDP不保证）  chapter_3_notes

你可能真正卡住的是：“分割”是谁做的？

- **UDP 不提供课件 rdt 那套“为了可靠而分段/重传/按序”。**  chapter_3_notes
- 如果应用一次发得太大，现实中通常会触发 **IP 分片**（或者发送直接报错，取决于系统/配置）。这部分很多课件在后面章节才细讲，你现在只要记住：
 - UDP 不会像 TCP 一样把“大数据流”管理成一串可控的 segment 序列（带序号、ACK、重传等）。  chapter_3_notes
 - 所以课件才会说：要在 UDP 上实现可靠传输，就得在应用层自己加可靠性（application-specific error recovery）。  chapter_3_notes

我反问你一个很关键的确认题（答完我们就继续 rdt2.0 的 fatal flaw）

如果 UDP 收到的 segment 坏了（checksum 检测出错），UDP 会像 rdt2.0 那样发 ACK/NAK 让对面重传吗？为什么？

来源

📄 👍 💬 ↗️ ↺️ ...

1. 不会，因为纯udp没有源头信息。那么rdt是有这个信息吗？我们说的rdt这一系列是什么定位？是一个与tcp、udp同等级的协议吗还是什么

已思考 27s >

你这里有两个“概念缠在一起”了：**UDP 会不会回 ACK/NAK**，以及 **rdt 这一串到底是什么地位**。我把它拆开讲清楚。

1) 你说“UDP 没有源头信息”——这点其实不对

课件在讲 demux 的时候明确说：每个 IP datagram 都有 **source/destination IP**，每个 TCP/UDP segment 都有 **source/destination port**，主机用这些信息把段交给正确 socket。 📄 chapter_3_notes

所以 UDP **当然有源信息**（源 IP + 源端口），否则你连 DNS 服务器都没法给客户端回包。

UDP 不回 ACK/NAK 的原因不是“没有源头信息”，而是：UDP 就是不提供可靠交付服务——它是 best-effort，可能丢、可能乱序；要可靠只能“在应用层自己加”。

📄 chapter_3_notes

2) 那为什么 UDP 不像 rdt2.0 那样做 ACK/NAK + 重传？

因为那一整套“可靠性机制”（差错恢复、反馈、重传、序号、处理重复包.....）本质上就是 TCP 的活儿。课件对 UDP 的定位就是 “no frills / bare bones”，并且明确说可靠性要应用层自己补。 📄 chapter_3_notes

所以在“课程的抽象”里你可以记：

- **UDP**: 提供 multiplex/demux + checksum (能检测错, 不负责恢复)
📄 chapter_3_notes
 - **TCP**: 在此基础上再提供 rdt 这套“可靠传输”能力 (后面 3.5 会具体讲)
-

3) rdt1.0/2.0/2.1/2.2... 到底是什么定位? 是和 TCP/UDP 同等级的新协议吗?

不是。rdt 不是一个互联网里真的“第三种传输层协议”。

它在课件里是一个“教学用的通用可靠传输协议 (reliable data transfer protocol)”的抽象模型, 用来演示:

在一个不可靠信道上, 如何一步步加机制得到可靠传输。 📄 chapter_3_notes

课件自己也说: 我们会 *incrementally develop* 这个 rdt, 用 FSM 描述, 先只考虑单向数据传输, 但控制信息 (ACK 等) 会双向流动。 📄 chapter_3_notes

你可以把它理解成:

- **rdt 系列** = “可靠传输的设计积木与推导过程” (checksum、ACK/NAK、seq#、duplicate discard...) 📄 chapter_3_notes
 - **TCP** = 把这些思想工程化后的真实协议 (当然更复杂: 窗口、超时、拥塞控制等)
-

4) “rdt 有源头信息吗? 为什么能发 ACK 回去?”

在 rdt 的抽象里, 接收方发 ACK/NAK 是通过 `udt_send()` 往下交给“不可靠信道”去送达。

📄 chapter_3_notes

它并没有否认地址信息的存在——现实里 ACK 回去靠的就是: 对方包里带的源 IP/源端口 (以及网络层路由)。课件把“怎么送到对方”这件事都封装进了 `udt_send()` 这个黑盒里。

我确认一下你现在的理解 (一个小题)

如果接收方发现 UDP checksum 不匹配 (error detected), UDP 会做什么?

- A) 发 NAK 要求重传
- B) 把坏的数据交给应用
- C) 丢弃这个 segment (可靠性由应用自己决定怎么补)

你选一个，我们就继续下一小节：**rdt2.0 的 fatal flaw**（ACK/NAK 也可能损坏）→ **为什么必须引入序号 rdt2.1/rdt2.2**。

来源



c;你没有给我过所谓demux的内容，demux是什么意思

已思考 23s >

选 C ☒ (UDP 检测到错一般就丢掉，不负责恢复)。

你问的 **demux 是什么意思**：它就是课件 3.2 这一节的核心概念——**分用 (demultiplexing)**。 chapter_3_notes

demux (demultiplexing, 分用) 一句话

接收端把收到的 **transport-layer segment**，按首部里的信息，交给“正确的 **socket** / 正确的进程”。 chapter_3_notes

和它配对的另一个词是 **mux (multiplexing, 多路复用)**：

- **发送端**：从多个 **socket** 收到数据 → **加上传输层首部** → 交给网络层发出去 (**mux**)
 chapter_3_notes
- **接收端**：从网络层拿到 **segment** → **用首部信息决定交给哪个 socket** (**demux**)

demux 具体“看什么信息”？

课件说：主机收到 **IP datagram**，每个 **datagram** 有源/目的 IP；里面的 **segment** 有源/目的端口。主机用 **IP 地址 + 端口号** 把 **segment** 定向到合适的 **socket**。 chapter_3_notes

UDP 的 demux：2-tuple

UDP socket 被 “(destination IP, destination port)” 这个二元组标识；只要目的 IP/端口相同，即使源 IP/端口不同，也会进同一个 socket。 chapter_3_notes

TCP 的 demux：4-tuple

TCP socket 用四元组标识：源 IP、源端口、目的 IP、目的端口；所以同一个服务器端口（如 80/443）能同时对应很多连接 socket。 chapter_3_notes

你确认一下（很短）

如果一台主机同时开着微信和浏览器，它们都在收数据：**demux** 做的事更像是

- A) 把多条应用数据“合并发出去”
- B) 把收到的数据“分发给正确的应用”

你选一个，我们就继续回到 3.4：讲 **rdt2.0** 为什么不够（ACK/NAK 自己也可能坏）→ 于是引入序号 **rdt2.1/2.2**。

来源



b

已思考 15s >

对，**demux** 就是接收端“分发给正确应用”（你选 B 正确）。  chapter_3_notes

现在我们回到 **3.4 Reliable Data Transfer** 里下一小节：**rdt2.0** 的致命缺陷 → 引入序号 **rdt2.1 / rdt2.2**。

rdt2.0 的 fatal flaw：ACK/NAK 自己也可能损坏

rdt2.0 假设“数据包可能被损坏”，所以我们有 checksum + ACK/NAK + 重传。

 chapter_3_notes

但课件马上问：如果 ACK/NAK 被损坏怎么办？（fatal flaw）  chapter_3_notes

为什么这会“致命”？

因为发送方收到一个“看不懂/损坏的 ACK/NAK”时，它不知道接收方到底发生了什么：

 chapter_3_notes

- 接收方其实收到了数据并交付了？
- 还是没收到 / 收到坏包没交付？

发送方此时“盲目重传”会导致一个新问题：可能产生 **duplicate**（重复交付）。

 chapter_3_notes

关键点：重复包本身不可怕，可怕的是“重复交付给上层应用”（比如银行转账扣两次）。

解决思路：加 Sequence Number，让接收方能丢弃重复包（rdt2.1）

课件给的处理 duplicates 的方案是三步：  chapter_3_notes

1. 发送方：ACK/NAK 损坏时，重传当前包
2. 发送方：给每个包加序号 seq#
3. 接收方：发现 duplicate 就丢弃（不 deliver up）

而且 seq# 用 0/1 两个就够（交替位 / alternating bit），因为这是 stop-and-wait：同一时刻最多只有一个“在路上”的数据包。  chapter_3_notes

rdt2.1 的直觉（不用死磕 FSM 也能做题）

- 发送方发 pkt0 → 等 ACK0
- 如果 ACK0 正常：发 pkt1
- 如果 ACK0 损坏/NAK：重发 pkt0
- 接收方那边永远“记住我现在期待的是 0 还是 1”，如果来了个不是期待的 seq，就当作重复包丢掉。

rdt2.2：去掉 NAK（只用 ACK 也能实现同样功能）

课件说 rdt2.2 和 rdt2.1 功能相同，但只用 ACK：  chapter_3_notes

做法是：

- 接收方如果收到坏包，不发 NAK
- 而是重复发送“上一个正确收到的包”的 ACK（带序号）
- 发送方一旦看到“重复 ACK”（或 ACK 序号不对），就把它当作 NAK 的效果：重传当前包

这就是很多网络协议里“duplicate ACK 触发重传”的思维来源。

我确认你真的理解了（两题很关键）

1. 为什么 seq# 用 0/1 就够？（提示：stop-and-wait 的“在途包”数量）  chapter_3_notes
2. 在 rdt2.2 里：接收方收到坏包时，会回什么？它起到的效果等价说成一句话是什么？

你答完如果没问题，我们就继续下一节 rdt3.0：信道还会丢包 → 引入 timeout 定时器（这节非常常考）。

[来源](#)

1. 因为途中永远只有一个包
2. 回上一个包的ack。有问题：
默认checksum不会被污染吗，如果第一个包就corrupt，
sender会等来什么信号？

已思考 36s >

你问得特别好，而且这个问题正好把 rdt2.2 的“默认 ACK”和 rdt2.x 的假设边界都暴露出来了。

1) checksum 会不会“被污染”？

会的——checksum 字段本身也属于 packet 的比特，也可能被信道翻转。课件里用 corrupt(pkt) / notcorrupt(pkt) 来抽象“通过校验和等差错检测判断是否损坏”。所以不是“checksum 永远可信”，而是：**接收方用 checksum 检测整包是否损坏。**

而且课件明确指出：rdt2.0 的致命缺陷就是 **ACK/NAK 也可能被损坏**。  chapter_3_notes

2) 如果“第一个数据包就 corrupt”，sender 会等来什么？

这里要分清：rdt2.2 的假设是“会 bit error，但不丢包”（丢包问题要到 rdt3.0 才处理）。所以在 rdt2.2 里，发送方不会“等不到任何东西”，接收方一定会回一个控制报文。

rdt2.2 的规则（关键一句）

接收方如果收到坏包（或收到不期望序号的包），**不发 NAK，而是发送“上一个正确收到的包”的 ACK（带序号）**；发送方看到“重复 ACK/错误序号的 ACK”，就重传当前包。

那“第一个包就坏了”，哪来的“上一个正确包”？

这靠一个约定的初始状态来解决：接收方一开始“期待 seq=0”，于是把“上一个正确收到的包”当成 seq=1（或理解成“ACK1 = 我还没正确收到 0”）。你在课件的 receiver fragment 里能看到：当期望 0，但收到 corrupt 或 has_seq1 时，它会回 ACK1（last pkt received OK）。

于是流程是：

1. sender 发 `pkt0`
2. receiver 收到但 `corrupt(pkt0) ⇒` 回 `ACK1` （表示“我最后正确的是 1/等价于我还没收对 0”）  `chapter_3_notes`
3. sender 收到 `ACK1` （不是期望的 `ACK0`） $\Rightarrow$ **重传 `pkt0`**  `chapter_3_notes`
4. 直到 receiver 正确收到 `pkt0`，才交付数据并回 `ACK0`

如果“ACK 也被 corrupt”呢？

rdt2.1/2.2 的 sender FSM 都把 `corrupt(rcvpkt)` 当成“出问题了”来处理：收到损坏的 ACK 就重传当前包。

如果你这块 OK 了，我们下一节就该进 **rdt3.0**：信道还会丢包（数据包/ACK 都可能丢） $\Rightarrow$ 必须引入 **timeout 定时器**，这也是你刚才“sender 等不到信号怎么办”的真正答案所在。

来源



checksum机制里，会不会有同时坏了两处的情景，然后这两个错误cancel out彼此，反而查不出错误

会的，checksum（尤其是课里常讲的 16-bit one's-complement checksum）确实可能“漏检”：出现多处错误，但它们在“加和”意义上刚好抵消，导致最终校验和不变，于是接收端以为没坏。

为什么会“抵消”？

one's-complement checksum 本质是在算：

1. 把报文按 16-bit 分组做加法（带回卷进位，end-around carry）
2. 取反得到 checksum

接收端把“所有 16-bit（含 checksum）”再加一遍，如果结果是全 1，就认为没错。

所以只要错误让“整体加和”保持不变（在这个模加系统里），就会漏检。

一个直观例子（概念级）

假设报文里有两个 16-bit 字：

- 原来： $w1 = 0x1000$ ， $w2 = 0x2000$ ，总和是 $0x3000$

- 传输错误后刚好变成: $w1' = 0x1001$ (+1), $w2' = 0x1FFF$ (-1)

两处错都真实发生了, 但总和还是 $0x3000$, 因此 checksum 可能仍然匹配 → 检测不出来。

(更一般地: 如果多处 bit flip 在数值上造成的改变量相加为 0, 就可能漏检。)

那这种漏检常见吗?

- 可能发生, 但概率通常很低: 如果错误近似“随机”, 16-bit checksum 的漏检概率大约在 $1/2^{16}$ 量级 (不是严格保证, 只是直觉)。
- 但如果错误模式很“结构化” (比如某些链路/硬件造成成对翻转), 漏检风险会比随机模型更糟。

这也是为什么很多地方更喜欢 CRC

链路层常用 **CRC**, 对“突发错误 (burst errors)”检测能力更强; checksum 更像是“轻量、便宜”的端到端兜底。

如果你愿意, 我可以顺着课件再讲一句: 为什么 rdt/TCP 还要 ACK/重传, 而不是“只靠 checksum”就完事 (核心就是: checksum 只能检测, 不能恢复)。



继续课件

已思考 31s >

好, 继续课件 3.4 的下一小节: rdt3.0: 信道既会出错也会丢包 (data/ACK 都可能丢)。

rdt3.0: channels with errors and loss (Stop-and-Wait + 定时器)

1) 新问题: 如果“包或 ACK 丢了”, 发送方会一直等

rdt2.2 只解决了“比特错误”, 但没解决丢包。rdt3.0 增加的新假设就是: 底层信道还可能丢包 (数据包、ACK 都可能丢)。  chapter_3_notes

所以仅靠 checksum/序号/ACK/重传还“不够”, 因为你可能根本收不到任何反馈。

 chapter_3_notes

2) 关键补丁: 发送方加一个倒计时定时器 (timeout)

rdt3.0 的思路非常直：发送方等一个“reasonable amount of time”。  chapter_3_notes

- 在时间内收到正确 ACK：继续发下一个新包
- 超时仍没收到 ACK：认为丢了 $\Rightarrow$ 重传当前包

注意课件也强调：如果只是“延迟”不是“丢”，超时重传会产生 duplicate，但序号机制已经能处理重复包（接收方不重复 deliver）。  chapter_3_notes

同时为了让发送方知道 ACK 在确认哪个包：ACK 必须显式带上被确认的 seq#。

 chapter_3_notes

3) 发送方 FSM 你怎么抓（不用死背图）

rdt3.0 仍然是 stop-and-wait：任意时刻最多 1 个 in-flight 包。  chapter_3_notes

因此发送方就两个“发包状态”反复切换：等 ACK0 / 等 ACK1；每次发完就 start_timer，收到正确 ACK 就 stop_timer；超时就重发并重启 timer。

4) 课件示例：ACK 丢失也没事（会重传，但不会重复交付）

课件的“rdt3.0 in action”里就画了 ACK loss：ACK 丢了 $\rightarrow$ sender timeout $\rightarrow$ 重传 pkt $\rightarrow$ receiver 发现是 duplicate（用 seq#） $\rightarrow$ 重新回 ACK。  chapter_3_notes

这就是 rdt3.0 能正确工作的核心：“可能重复传输，但不会重复交付”。  chapter_3_notes

5) 但性能很差：Stop-and-Wait 利用率 (utilization) 极低

课件马上转向“为什么还要更复杂的协议”：rdt3.0 正确但性能 stink。  chapter_3_notes

它给了一个 1Gbps 链路、单程传播 15ms、1KB 包的例子：

- $RTT = 30ms$
- 发送 1KB 的传输时间 L/R 只有几微秒量级
- 所以发送端大部分时间都在等 ACK，利用率 U 非常小（课件算到了极小的数值，并指出吞吐远低于链路带宽）。  chapter_3_notes

这就是下一页引出 pipelining（流水线）的动机：别一次只飞一个包。

你确认理解的小问题（答完我们就进 3.4 后半：pipelining $\rightarrow$ GBN/SR）

1. 为什么 rdt3.0 里“超时重传”不会造成应用层重复收到同一份数据？（用 seq#/duplicate 的语言回答）

2. 如果一个包只是延迟很久（没丢），但超过 timeout 才到，会发生什么？系统靠什么保证最终仍正确？

来源



1. 因为这是基于 rdt2.2 的序号机制，如果接收方收到了同一个包但是序号不是它当前排的（因为重传的包序号不会 flip），会丢弃
2. 会重传，但是接收方不会接

已思考 16s >

你的 1) 答得很准；2) 我帮你补全一下：

- 会重传 ✓
- 接收方不是“不接”：它可能会收到“迟到的原包”和“重传包”，但会用 **seq#** 识别重复，不重复 deliver 给应用，并且通常会把 ACK 再发一次（让发送方尽快停止重传）。这正是 rdt3.0 里“延迟≠丢失”那条注释想表达的：会产生 duplicate，但序号机制能处理。

📄 chapter_3_notes

下面继续课件 3.4 后半：Pipelined protocols（流水线）——也就是为什么 stop-and-wait 性能差，以及怎么改。 📄 chapter_3_notes

Pipelining：一次让多个包“在路上”

pipelining 的定义：发送方允许同时有多个“已发送但未被 ACK”的包在网络中飞着（in-flight）。 📄 chapter_3_notes

为什么能显著提升利用率（utilization）？

stop-and-wait 的痛点是：发完 1 个包就傻等 RTT；链路大部分时间空着。课件画了 3-packet pipelining 的时间轴：如果你能连续发 3 个包再等 ACK，发送方忙于发送的比例就大致提升 **3 倍**。 📄 chapter_3_notes

（本质：把“等待 RTT 的空档”用后续包的发送填起来。）

但代价是什么？

课件直接点了 2 个代价： 📄 chapter_3_notes

1. **sequence number 范围必须变大**（因为同时在途的不止 0/1 两个）
2. **需要缓冲 (buffering)**：发送方要缓存未确认的包；接收方可能也要缓存乱序包

两种通用的流水线协议：GBN vs SR

课件说有两类“generic forms”：**Go-Back-N (GBN)** 和 **Selective Repeat (SR)**。

📄 chapter_3_notes

总览对比（先记最关键三条）

GBN: 📄 chapter_3_notes

1. 发送方窗口内最多 **N** 个未 **ACK** 包
2. 接收方只发 **cumulative ACK**（累计确认）：如果中间有“缺口”，后面的就不确认
3. 发送方只给最老的未确认包设一个 timer；一旦超时 → 从最老那个开始把后面全重传

SR: 📄 chapter_3_notes

1. 发送方窗口内最多 **N** 个未 **ACK** 包
2. 接收方对每个正确收到的包都发 **individual ACK**（逐个确认）
3. 发送方对每个未确认包各自维护 timer；超时 → 只重传那个包

一句话：**GBN 省事但可能重传很多；SR 更精细但需要更多状态/缓存。**

我先停在这里，问你两个“课件式”检查题

1. 你用一句话解释：**GBN 的 cumulative ACK(n)** 表示什么？（课件原句是“到 n 为止都正确收到”） 📄 chapter_3_notes
2. 你直觉上觉得：如果网络经常丢单个包、但后续包大多能到，**GBN 还是 SR 更省重传？为什么？** 📄 chapter_3_notes

你答完我再继续讲下一页：**GBN 的 sender/receiver 行为**（窗口、expectedseqnum、不缓存、丢弃乱序），以及“GBN in action”那张图怎么读。

来源



1. sender的窗口允许缓存n个，按顺序出栈（按顺序收ack），如果有一个本应先收ack的没收到就来了后面的，也

不算；如果timer过了，这个漏洞之后的全部重传2. sr 问题，如果gbn的时候，一个漏了后面都来了，然后timer 内这个也来了，那么前面来的、这个位置后面的包的ack会被记忆和接受吗

已思考 41s >

你这个追问正好击中 **GBN 和 SR 的本质差别：GBN 接收方“不记忆乱序包”**。

先把你的 1) 再“课件化”一句

GBN 的 **cumulative ACK(n)** 的定义是：序号 $\leq n$ 的包都已经按序正确收到。

📄 chapter_3_notes

所以当中间有缺口时，接收方不会 ACK 缺口后面的那些包（哪怕它们到达了）。

📄 chapter_3_notes

回答你的问题：GBN 中“漏了一个，后面都来了；但漏的在 timer 内也来了”，后面的包/ACK 会被记住吗？

不会。

GBN 接收方操作很“抠门”：只记一个 `expectedseqnum`，不做接收端缓存（no receiver buffering）。

规则是：

- 收到 **按序且正确** 的 `pkt n`：交付上层 + 回 `ACK(n) + expectedseqnum++`
📄 chapter_3_notes
- 否则（乱序 or corrupt）：丢弃该包，并重发“最近一次按序收到的包”的 ACK

用你说的情景走一遍（最典型：pkt2 丢，pkt3/pkt4/pkt5 都到了）

- receiver 已经按序收到了 `pkt0`、`pkt1`，所以 `expectedseqnum=2`，最近按序 ACK 是 `ack1`
- `pkt3` 先到：receiver **discard** `pkt3`，并 **(re)send** `ack1` 📄 chapter_3_notes
(注意：`pkt3` 并没有被“先存起来”等 `pkt2` 来了再补交付——GBN 明确“不缓存”)
📄 chapter_3_notes
- 后来 `pkt2` 在 timer 内到达：receiver 这时会 **deliver** `pkt2` 并回 `ack2`，
`expectedseqnum` 才能推进 📄 chapter_3_notes

关键：`pkt3/pkt4/pkt5` 之前那次“到达”已经被丢掉了，所以它们不会因为 `pkt2` 来了就“自动被记起来”。要想交付 `pkt3/pkt4/pkt5`，必须让它们再到一次（通常靠发送方后续重传）。 📄 chapter_3_notes

那发送方会怎么做？会不会“永远等不到 ack3”？

不会永远卡死，但会 **晚一点**：

- 发送方一旦收到 ack2，它的 send_base 会推进到 3，并把 timer 移到最老未确认包 (pkt3) 上  chapter_3_notes
- 由于 pkt3 之前到过但被 receiver 丢了，receiver 不可能发出 ack3
- 所以等 pkt3 的 timer 超时后，GBN 会从 send_base 开始把未确认的都重传 (pkt3、pkt4、pkt5...)

这也是为什么在“单包丢失但后续都能到”的网络里，**SR 更省重传**：SR 会把 pkt3/pkt4 缓存起来，等 pkt2 一到就能一口气按序 deliver。  chapter_3_notes

小检查题（答完我就继续讲 SR 的窗口/缓存机制）

在 GBN 里，当 receiver 收到一个乱序包 pkt3 时，它为什么要 **重复发送 ack1**？这个 ack1 试图向 sender 传递什么信息？

来源



不会永远卡死，但会 **晚一点**：

- 发送方一旦收到 ack2，它的 send_base 会推进到 3，并把 timer 移到最老未确认包 (pkt3) 上
- 由于 pkt3 之前到过但被 receiver 丢了，receiver 不可能发出 ack3
- 所以等 pkt3 的 timer 超时后，GBN 会从 send_base 开始把未确认的都重传 (pkt3、pkt4、pkt5...)

我说的不是这个。我的问题只看了 sender 处，假设 receiver 都正常发了 ack，但是 for some reason sender 处是 ack4, ack5, ack3 的顺序接受的

已思考 31s >

明白，你问的是 **ACK 在网络里乱序到达 sender**：比如 sender 先收到 ACK4，再收到 ACK5，最后才收到 ACK3。

在 GBN 里这完全 OK，因为 **ACK 是 cumulative (累计确认)**：ACK(n) 表示“序号 $\leq n$ 的包都已经被 receiver 正确按序收到了”。  chapter_3_notes

GBN sender 收到乱序 ACK 会怎么处理？

关键规则（课件 sender FSM）

只要收到一个没损坏的 ACK，sender 会做：

`base = getacknum(rcvpkt) + 1`（也就是把 `send_base` 直接推进到 `n+1`）

 chapter_3_notes

并且课件提醒：**may receive duplicate ACKs**（可能收到重复 ACK）。  chapter_3_notes

套到你的例子：ACK4 → ACK5 → ACK3

假设当前 `base=3`（在等 `pkt3` 的累计确认）：

1. 先到 ACK4

- 这表示 `pkt0..pkt4` 都已收（累计确认）
- sender 直接 `base = 4+1 = 5`，窗口整体右移到从 5 开始  chapter_3_notes

2. 再到 ACK5

- sender 再推进：`base = 5+1 = 6`

3. 最后到 ACK3

- 这时 ACK3 的含义比 sender 当前知道的还“落后”（因为 sender 已经把 0..5 都当作确认过了）
- 所以它就是 **duplicate/old ACK**，GBN 里会被 **忽略**（不可能让 `base` 往回退）

直觉：**累计 ACK 天生抗乱序**。你先收到更大的 ACK，本质上已经包含了小 ACK 的信息。

小补充：这种乱序为什么“逻辑上允许”？

在 GBN receiver 侧，确实只会对“最后按序收到的包”发累计 ACK。  chapter_3_notes

所以 ACK5 一定意味着 receiver 已经按序收到了 0..5；ACK3 只是网络里更早发出的 ACK 过了一会儿才到 sender（乱序/延迟），因此变成“过期重复 ACK”。

如果你这块没问题，我们下一步继续课件：**SR (Selective Repeat)** 为什么需要对每个包单独 ACK + receiver buffering，以及它如何处理“乱序到达的数据包”。

来源



有问题，我不认可这里sender收到ack4就代表前面也收到了。假设本来的情景是按顺序收345，但是他们按照顺序从receiver发出后，for some reason收到的顺序是453，这时sender会怎么做，因为这样的话整体的sender接收到的ack顺序就是2453

已思考 21s >

你这里“不认可 ACK4 代表前面也收到了”的直觉，在 SR（逐个 ACK）里是对的，但在 GBN（累计 ACK）里是不成立的。关键就在课件对 GBN 的定义：ACK 是 cumulative。

在 GBN 里，为什么 receiver 能发 ACK4 就意味着 0..4 都收到了？

因为 GBN receiver 的规则是：

- 只要收到“期望的那个序号”并且不 corrupt，才 deliver，并把 `expectedseqnum++`
- 如果收到乱序包（比如缺 3 先来了 4），它会 **直接丢弃**，并且回的是“最后一个按序收到的包”的 ACK（也就是重复 ACK）

所以：只要 receiver 还在等 3，它就不可能发 ACK4。

它最多发 ACK2（重复）来表示“我还缺 3”。

你给的 ACK 到达顺序：2, 4, 5, 3（ACK3 延迟很久才到）

这在 GBN 里完全可能发生，而且 sender 的处理是：

- 收到 ACK2：base 变成 3（或保持）
- 收到 ACK4：因为是累计 ACK，表示 0..4 已按序收到 $\Rightarrow$ base 直接跳到 5
- 收到 ACK5：base 跳到 6
- 最后收到 ACK3：这是“过期/重复 ACK”，忽略（base 不会往回退）

注意：这里 sender **不需要**先收到 ACK3 才能“相信”3 被收到了；因为在 GBN 语义里，ACK4 本身就包含了 ACK3 的信息（不是“选择性确认”，而是“最后按序到哪儿了”）。

什么时候你这种“ACK4 不代表 ACK3”才成立？

那是 SR (Selective Repeat) / 或 TCP 的 SACK 类似思想：ACK 是“逐包确认”，ACK4 只说明 4 收到了，不保证 3 收到。GBN 不是这样。

给你一个反证，帮你彻底放心

如果真的发生“pkt3 丢了，但 pkt4/pkt5 到了”，GBN receiver 会丢弃 4/5 并重复发 ACK2，**绝不会产生 ACK4/ACK5**。

你要不要我接着按课件把 SR 的 receiver buffering 和“窗口重叠/序号空间”那个最容易混的坑讲掉？（SR 里你刚才的直觉就会变成正确的。）



我说的是gbn的情况下，receiver按顺序顺利收到了pkt2345，然后receiver发出ack2345，但是sender收到的时候是ack2453，是这种情况不会发生所以不考虑，还是怎么样

已思考 30s >

这种情况**会发生**，因为 ACK 本质上也是在网络里传的分组，可能被**延迟/乱序**。GBN 不求 ACK 按序到达；它靠 **cumulative ACK** 的语义来“抗乱序”。  chapter_3_notes

你设定的场景：receiver 按序收到了 pkt2,3,4,5，并依次发 ACK2, ACK3, ACK4, ACK5

但 sender 收到顺序是 ACK2 → ACK4 → ACK5 → ACK3（乱序）

GBN sender 的规则（课件给的 sender 扩展 FSM）

收到一个没损坏的 ACK 时，直接做：

- `base = getacknum(rcvpkt) + 1`

并且课件也明确提醒：**may receive duplicate ACKs**。  chapter_3_notes

所以处理过程是：

1. 收到 ACK2

- `base = 3`

2. 收到 ACK4 (虽然 ACK3 还没到)

- 因为是 **cumulative ACK(4)**，它的含义就是“到 4 为止都正确收到了”  chapter_3_notes
- 所以 sender 直接 `base = 5` (相当于“ACK3 被包含在 ACK4 里”)

3. 收到 ACK5

- `base = 6`

4. 最后收到 ACK3

- 这是一个“旧的/重复的 ACK”，不会让 `base` 往回退，等价于 **ignore duplicate ACK** (课件在 GBN in action 图里也写了这点)。  chapter_3_notes

定时器会怎样？

GBN 只给“最老未确认包 (`send_base`)”设 timer；当 `base` 推进时，timer 也跟着停/重启 (课件 FSM 里写了 `base` 更新后 `stop_timer` / `start_timer` 的逻辑)。  chapter_3_notes

一句话总结

在 GBN 里：ACK 乱序到达没关系，因为 大的 **cumulative ACK** 覆盖了小的 ACK；后到的小 ACK 只是“过期/重复 ACK”，忽略即可。

你这块 OK 之后，我就继续课件下一节：**Selective Repeat (SR)** ——因为你刚才这种“ACK4 不代表 ACK3”的直觉，其实就是 SR/TCP-SACK 那一类协议的世界观。

来源



懂了，继续

已思考 57s >

好，继续课件。我们先过这一节：**Selective Repeat (SR, 选择重传)**，只讲这一节，讲完我会留一个小题让你确认你真的懂了，再决定要不要继续下一节。

3.4 可靠传输：Selective Repeat (SR) 在解决什么问题？

在 GBN 里：只要中间丢/坏一个包，后面即使都到了也要“回退重传一串”；因为接收方只认“按序”，不缓存乱序（gap 后面的全丢）。

SR 的核心目标就是：

允许流水线 + 允许乱序到达 + 接收方缓存乱序包 + 发送方只重传丢的那一个。

📄 chapter_3_notes

课件给的 SR 总结（你可以当成三条“信仰”背下来）：

1. sender window：最多同时飞 N 个未确认包
2. receiver：每个正确收到的包都单独 ACK，必要时缓存，最后按序交付
3. sender：每个未确认包各自一个 timer，谁超时重传谁

📄 chapter_3_notes

SR 的“窗口”到底在干嘛？（把规则说清楚）

发送方 (sender) 侧规则（对应课件伪代码/状态机）

- 上层给数据：如果窗口里还有“可用序号”（蓝色区域），就发出去并启动该包的 timer。
- 收到 ACK(n)（注意 SR 是 *individual ack*）：只要 ACK 的序号 n 落在 sender 窗口里，就把 pkt n 标记为“已确认”。
 - 但 `send_base` 只会推进到“最小的未确认包”（也就是 `base` 指着“最老的洞”）。

📄 chapter_3_notes

- `timeout( $n$ )`：只重传 pkt n （不会像 GBN 那样从 `base` 开始全重传）。

📄 chapter_3_notes

你刚才纠结的“ack 顺序乱了怎么办”：

在 SR 下这完全正常：收到 ACK4、ACK5 先到都没问题——先把 4、5 标绿；但 `base` 如果还卡在 3，就不会滑动，直到 ACK3 到来才会把窗口整体向右滑。

这正是 SR 相比 GBN 的关键差异：SR 允许“洞后面的确认先到”，只是不会让窗口越过那个洞。

📄 chapter_3_notes

接收方 (receiver) 侧规则（这块最关键：为什么要缓存）

课件把接收方分成三个区间（你可以当作“收包判定三分法”）：

1. pkt n 在当前接收窗口 $[rcvbase, rcvbase + N - 1]$ ：
 - 立刻 ACK(n)
 - 如果乱序：先 `buffer`
 - 如果刚好是 `rcvbase`（按序到达）：交付，并且顺便把 `buffer` 里连续的也一并交付，然后滑动 `rcvbase`

📄 chapter_3_notes

2. pkt n 在“上一个窗口” $[rcvbase - N, rcvbase - 1]$:

- 这通常表示“这是重复包”（比如 ACK 丢了导致 sender 重传）
- receiver 会再 ACK(n) 一次（告诉 sender：我早收到了） [chapter_3_notes](#)

3. 其他情况：直接 ignore [chapter_3_notes](#)

用你提的 ACK 乱序例子，SR 会怎么走？

你给的情景可以套进 SR 的 sender 规则：

- sender 已经发了 pkt2,3,4,5（窗口 $N=4$ ，base=2）
- receiver 都收到了，于是发 ACK2,3,4,5
- sender 收到顺序是 ACK2, ACK4, ACK5, ACK3

SR sender 的反应：

- 收到 ACK2：pkt2 标绿，base 从 2 试着往右滑，但发现 pkt3 还没绿，所以 base 停在 3
- 收到 ACK4：pkt4 标绿（允许乱序确认）
- 收到 ACK5：pkt5 标绿
- 收到 ACK3：pkt3 标绿，此时 base 才能从 3 一路滑过 4、5（因为它们早绿了），窗口整体右移

这就是 SR 的设计初衷：确认可以乱序到，窗口不会被“洞后面的确认”误导，但也不会浪费重传。 [chapter_3_notes](#)

你来答一个 10 秒小题（确认你真的吃透 SR）

SR，发送窗口 $N=4$ 。此刻 sender 的 $send_base=3$ ，它已经发出了 pkt3、pkt4、pkt5、pkt6。

如果 sender 先收到 ACK5（但 ACK3、ACK4 还没到）， $send_base$ 会变成多少？为什么？

你回答完我再继续下一节（TCP 概览 / TCP segment 结构）。

来源



不变还是3，因为base要变要一个一个变

对✅： `send_base` 仍然是 3。因为 SR 的窗口滑动必须从 base 开始连续“补齐”，ACK5 只能把 pkt5 标记已确认，但不能让 base 跨过还没确认的 pkt3/pkt4。 [chapter_3_notes](#)

继续课件下一节：3.5 Connection-oriented transport: TCP（面向连接的传输：TCP），我先讲“TCP 提供什么 + 连接怎么被唯一标识 + segment 长什么样”，不一次讲拥塞控制。

3.5 TCP：它到底“比 UDP 多提供了什么”？

课件把 TCP 的服务概括成几类（你可以当作考试答题模板）： [chapter_3_notes](#)

1. 面向连接（connection-oriented）：通信前要建立连接（握手），通信后要拆除
2. 可靠、有序（reliable, in-order byte stream）：应用看到的是“按序字节流”，丢包/乱序都由 TCP 处理
3. 流量控制（flow control）：别把接收方缓冲撑爆
4. 拥塞控制（congestion control）：别把网络撑爆（后面章节细讲）

同时 TCP 是 **full-duplex**（双向同时传数据），且点对点（一个连接两端）。

[chapter_3_notes](#)

TCP 连接怎么被“识别”？

TCP 的 demux 是靠 **4-tuple**：源 IP、源端口、目的 IP、目的端口。 [chapter_3_notes](#)

所以同一个服务器端口（比如 80/443）可以同时服务很多客户端连接，每个连接都有不同的源端口/源IP组合。

TCP segment 结构（你需要先认识字段）

课件的 TCP 首部图里，最关键要先认识这些： [chapter_3_notes](#)

- **source port / dest port**：给 demux 用
- **sequence number**：这个 segment 的数据在字节流里的位置
- **acknowledgment number**：我“期望收到的下一个字节”的序号（累计确认的味道）
- **flags**：尤其是 ACK、SYN、FIN（连接建立/拆除会用）
- **receive window (rwnd)**：流量控制用

- **checksum**: 差错检测

注意 TCP 的序号是按 **字节流 byte** 编号，不是按 “packet 编号”。这点是 TCP 区别于我们前面 rdt/GBN/SR（按包编号）的关键。  chapter_3_notes

你先回答一个很短的问题（我用来决定下一小节讲 seq/ACK 的具体例子）

TCP 里为什么说它提供的是 **byte stream**，所以 sequence number 要按“字节”而不是按“包”编号？你用直觉解释一句就行。

来源



不懂

好，我用“你在写文件/读文件”的直觉把它讲明白。

为什么 TCP 要按“字节”编号，而不是按“包”编号？

因为 TCP 对应用提供的抽象不是“一个个消息包”，而是一条连续的字节流（**byte stream**）：应用只看到“我写进去了一串字节，对方按顺序读出同样的字节”。

但在网络里，这串字节必须被 TCP 拆成若干个 **segment** 去发，而且：

- 每个 segment 可能大小不同（受 MSS、拥塞窗口、应用写入大小影响）
- 某个 segment 可能丢了，需要重传
- ACK 也常常是“累计确认”（确认到某个字节为止）

所以如果你只给“第几个包”编号，会马上遇到麻烦：

丢了一个包、或者拆包大小变了，“包编号”就没法精确表达“我现在缺的是哪一段内容”。

一个具体例子（最关键）

应用发送 3000 字节数据。TCP 可能拆成：

- segment A: 字节 1–1000
- segment B: 字节 1001–2000
- segment C: 字节 2001–3000

如果 B 丢了，接收方其实缺的是 1001–2000 这 1000 个字节。

用“字节序号”就能非常精确地说：

- ACK=1001 意味着：“我下一个想要的字节是 1001”（也就是 1–1000 都收到了）

 chapter_3_notes

重传时无论你把 B 重新打包成 1 个大段、还是拆成两个小段，只要它们覆盖字节 1001–2000，接收方都能拼回原始字节流。

这就是 **byte stream + byte-based seq/ack** 的意义：让 TCP 能在“分段大小可变、可能重传、可能乱序”的现实下，仍然给应用提供“连续有序字节流”的体验。

我确认一下你现在是否通了（只选一个回答就行）：

如果接收方回 ACK = 2001，它的意思更接近：

A) “我收到了第 2001 个包”

B) “我已经按序收到了 到 2000 字节为止，下一个要 2001”  chapter_3_notes

来源



B

对，就是 B ：ACK=2001 表示“到 2000 字节都按序收到了，下一个想要 2001”。

 chapter_3_notes

继续课件里 TCP 的下一小块：TCP 的 Seq / ACK 到底怎么配合工作（用一个最典型的例子讲清楚）。

TCP 的 Seq/ACK：最常用的“累计确认”直觉

1) Seq number：这个 segment 携带的数据从“第几个字节”开始

如果一个 segment 的 Seq = x，并且它携带了 L 个字节的数据，那么它覆盖的字节范围就是：

来源



pause, 那这跟udp/rdt是不同的吗, 他们不是字节流吗

对, 这点确实是 TCP 跟 UDP/rdt 最核心的不同之一。

TCP: 是“字节流 (byte stream)”

- 对应用来说, TCP 像一根管子: 你 `write()` 进去一串字节, 对方 `read()` 出来同样顺序的一串字节。 **没有“消息边界”**这个概念。
- 所以 TCP 的 **Seq/ACK** 是按“字节位置”编号, 用来精确表达“我收到了字节流的哪一段”。 [chapter_3_notes](#)

UDP: 是“数据报 (datagram) /消息边界保留”

- UDP 对应用更像“发一条消息就到一条消息”: 一次 `sendto()` 对应一次 datagram; 接收端一次 `recvfrom()` 读到的就是一整个 datagram (如果被截断)。
- 它不保证可靠/有序, 只提供很轻量的传输。

rdt (rdt1.0/2.x/3.0/GBN/SR): 是课件里的“教学抽象”

- rdt 系列是在讲“如何在不可靠信道上实现可靠传输”的通用设计积木, 所以课件里把单位讲成 packet/segment, 并用 seq# (常按包) 来说明重传、乱序、重复包处理。
- **真实 TCP** 把这些思想工程化了, 但由于它提供的是 byte stream, 所以序号改成“按字节编号”。 [chapter_3_notes](#)

如果你这块 OK, 我就接着把刚才那句补完:

“如果一个 TCP segment 的 Seq=x, 携带 L 字节, 那么它覆盖的字节范围是”

来源



那udp的datagram常常是应用的一次message吗, 还是说也涉及分割

通常来说: 是的, UDP 的一个 datagram 往往就是应用“一次 send/sendto 发送的一条 message” (消息边界被保留)。

但要加两层现实世界的“分割”细节：

1) UDP 层本身不负责“把一条大消息拆成多个 UDP datagram”

- 你 `sendto()` 多少字节，UDP 就尽量把它当作一个 UDP datagram 发出去。
- 接收端 `recvfrom()` 也是按“一个 datagram 一次交付”来给你；如果你提供的接收缓冲区太小，可能会截断这条 datagram（剩下的不会在下次 `recvfrom` 继续给你）。

2) 但“更底层”可能发生 IP 分片（fragmentation）

如果你一次 `sendto()` 发得太大，超过路径上的 MTU：

- 要么在网络层发生 **IP fragmentation**：一个 UDP datagram 会被拆成多个 IP 分片在网络里走；接收端 IP 层再重组回完整 datagram，才交给 UDP。
- 要么如果路径/系统设置不允许分片（常见于某些配置/IPv6 场景），就可能直接失败（例如返回 “Message too long” 之类）。

这也是为什么很多基于 UDP 的应用（DNS、音视频、QUIC 等）会自己在应用层做分片/分块，控制每个 datagram 的大小，尽量避免依赖 IP 分片（分片丢一个就整条 datagram 失败，代价很大）。

一句话总结：

- **UDP**：message 边界保留；一次发送~一个 datagram
- 分割如果发生，通常是应用层自己做，或由 IP 层被迫分片，而不是 UDP 负责



继续

已思考 18s >

好，继续到课件 3.5 TCP: Seq numbers + ACKs（这一小节），把“字节流编号”讲透。

 chapter_3_notes

TCP Seq number：按字节编号，不是按包编号

课件先给两个定义： chapter_3_notes

- **MSS (Maximum Segment Size)**：一个 TCP segment 里最多能装多少 **数据字节**（不含头）。
- **Sequence number for a segment**：这个 segment 里**第一个数据字节**在整条字节流里的编号。

所以如果某段的 $\text{Seq} = x$ ，并且它携带了 L 字节数据，那么它覆盖的字节范围是：

- $x, x+1, \dots, x+L-1$

课件举例：文件 500,000 字节， $\text{MSS}=1000$ ：

- 第 1 个 segment: $\text{Seq}=0$
- 第 2 个 segment: $\text{Seq}=1000$
- 第 3 个 segment: $\text{Seq}=2000 \dots$ [chapter_3_notes](#)

TCP ACK number: 我“下一步想要的字节”

课件定义：接收方放在 ACK 里的数字 = **next byte expected**（我期望收到的下一个字节序号）。 [chapter_3_notes](#)

两个特别关键的例子（强烈建议你背成直觉）：

1. Host A 已经收到了来自 B 的字节 **0..535**

→ A 回 $\text{ACK} = 536$ [chapter_3_notes](#)

2. Host A 收到了两个 segment：

- 一个包含字节 **0..535**
- 另一个包含字节 **900..1000**（中间缺 536..899）

→ A 仍然回 $\text{ACK} = 536$ ，因为 TCP 的 ACK 是 **cumulative**：确认到“第一个缺失字节”之前为止。 [chapter_3_notes](#)

课件还补了一句很现实的话：“接收方怎么处理乱序段，TCP 规范没规定，取决于实现”。也就是说：实现可以选择缓存乱序，也可以先丢弃，但“ACK 的语义”仍然是累计到第一个缺口。 [chapter_3_notes](#)

10 秒小题（你答完我继续讲下一页：telnet 场景 Seq/ACK 怎么跳）

一个 TCP segment: $\text{Seq} = 1000$ ，携带 $L = 200$ 字节数据（所以覆盖 1000..1199）。如果接收端之前已经按序收到了 **0..999**，那么它回的 ACK 应该是多少？为什么？

来源



对✅：回 **ACK=1000**。因为它期望的“下一个字节”就是 1000（也就是确认了 0..999 都按序到了）。  chapter_3_notes

继续课件下一小节：**Telnet example: Seq/ACK 在交互式输入里怎么变化**（这页常用来训练你“ACK=next expected byte”的直觉）。  chapter_3_notes

Telnet 例子要你看懂什么？

Telnet 是“每敲一个键就发一个很小的数据段”的交互式应用。课件用它说明两件事：

 chapter_3_notes

1. Seq/ACK 都按字节跳（哪怕一次只发 1 byte）
2. ACK 通常是双向的：我在发数据时，也会顺便 piggyback 我对你数据的 ACK（如果有要确认的东西）

怎么读这张图（你看图时记住这三条）

- 每条箭头上会标：Seq=... , ACK=... , data=...
- Seq：我这次发的第一个数据字节编号
- ACK：我“下一步想收到你的哪个字节”

我先问你一个图理解题（你不用贴图，口头规则回答）

在 Telnet 这种“每次发 1 byte”的场景里：

如果某一端刚刚发送了 1 byte 数据，那么它下一次发送数据时 Seq 会怎么变化？（加多少？）为什么？

来源



什么是piggyback

****piggyback（捎带 ACK）**就是：**

在 TCP 里，如果我本来就要给你发数据段（segment），那我就把“对你之前发来的数据的 ACK”顺便放在这个数据段的 ACK 字段里一起发走，而不是另外再单独发一个纯 ACK 段。

直觉类比：我本来就要寄一封信给你，于是把“已收到你上封信”的回执写在这封信里一起寄。

它的好处：减少额外包、降低开销（尤其是交互式场景里 ACK 很频繁）。



加8，说明这是下一个字节开始的

已停止思考 >

应该加一

对，应该加 1 （你第二条修正是对的）。

因为 TCP 的序号是按字节编号：你这次发送了 1 byte，那么下次你自己发送数据时，Seq 就向前推进 1，指向“下一次要发送的那个字节”。  chapter_3_notes

（你说的“加 8”更像是“发了 8 个字节”才会发生的情况。）

继续课件下一小节：TCP 的可靠性机制在工程上怎么实现：超时重传（timeout / retransmission）+ RTT 估计。这是 TCP 里 rdt3.0 思想的现实版本。

TCP Retransmission：为什么必须估计 RTT？

TCP 需要一个 timeout 来判断“可能丢包了，该重传了”。

但网络 RTT 会变化，如果 timeout 设得：

- 太小：会过度重传（其实只是延迟）
- 太大：丢包恢复太慢

所以课件引入：用样本 RTT（SampleRTT）估计平均 RTT（EstimatedRTT），再估计抖动（DevRTT），最后得到超时阈值（TimeoutInterval）。

课件公式（你只要记住“指数平滑 + 加安全裕度”）

1. EstimatedRTT（指数加权移动平均）：  chapter_3_notes

$$EstimatedRTT = (1 - \alpha) \cdot EstimatedRTT + \alpha \cdot SampleRTT$$

课件常用 $\alpha = 0.125$ 。

2. DevRTT (RTT 波动的指数平滑): [chapter_3_notes](#)

$$DevRTT = (1 - \beta) \cdot DevRTT + \beta \cdot |SampleRTT - EstimatedRTT|$$

课件常用 $\beta = 0.25$ 。

3. TimeoutInterval: [chapter_3_notes](#)

$$TimeoutInterval = EstimatedRTT + 4 \cdot DevRTT$$

直觉: EstimatedRTT 是“平均水平”, 4*DevRTT 是“留出波动的安全边际”。

小题 (用来确认你能用“直觉”记住这页)

如果网络突然开始更抖 (RTT 波动变大), 但平均 RTT 没怎么变, 那么上面哪一项会主要把 TimeoutInterval 拉大? 为什么? [chapter_3_notes](#)

来源

📄 🍷 🗨️ 📌 🔄 ...

pause, 课件中提到的receive window是什么

****receive window (rwnd) ****是 TCP 首部里的一个字段, 用来做 **flow control (流量控制)**: 告诉对方“我这边接收缓冲区还剩多少空间, 你最多还能在路上放这么多未被我应用取走的数据”。

直觉: 接收方在说: “我邮箱还剩这么多格子, 别再塞爆了。”

更具体一点 (你先抓住这两句就够用):

- **rwnd 是接收方通告给发送方的“可用缓冲”大小 (单位是字节)。**
- 发送方会用它限制自己“未确认且未被接收应用取走”的在途数据量, 避免把接收方的 buffer 撑爆。

如果你 OK, 我就继续回到你刚才问的那页: RTT 抖动变大时, 哪个量会把 Timeout 拉大。



那么这个window流动的规则跟上面提到的sr的rdt是一样的吗

不一样，但长得很像：TCP 里确实也有“窗口在滑动”，但它同时受 **两类窗口/规则** 约束，而 SR 的窗口主要是为“可靠性/选择重传”服务的。

SR 的窗口（课件 rdt 的 SR）

- 目标：可靠性 + 选择重传
- receiver 会缓存乱序包；ACK 可以是逐包 ACK；窗口滑动看“从 base 开始是否连续补齐”。

TCP 的窗口（现实 TCP）

TCP 的“能在路上的数据量”主要由两件事共同限制：

1. **rwnd (receive window)**：流量控制——别把接收方缓冲撑爆（你刚问的那个）。
2. **cwnd (congestion window)**：拥塞控制——别把网络撑爆（后面章节会讲）。

 chapter_3_notes

所以 TCP 发送方实际允许的 in-flight 数据量常写成：

$$\text{in-flight} \leq \min(\text{rwnd}, \text{cwnd})$$

（概念上这么理解就行。）

可靠性那部分，TCP 更像哪一个：GBN 还是 SR？

课件这章通常会强调：TCP 的 ACK 是 **cumulative**（像 GBN 的味道），但 TCP 实现往往也会缓存乱序（有点 SR 的味道），而且还有快速重传/选择确认等工程机制。你可以先记成：

- **TCP 基本 ACK 语义：更像 GBN（累计 ACK）**  chapter_3_notes
- **TCP 实际行为/优化：有 SR 风格的成分（缓存/选择性重传）**（课内如果不展开，你先用不用深挖）

我用一句话回答你的问题

rwnd 的“窗口流动”不是 SR 那套“选择重传窗口”的同一套规则；rwnd 是“容量通告/流量控制窗口”，而 SR 的窗口是“可靠性协议的发送/接收范围”。它们都叫 window，但“为什么滑、怎么滑”的驱动力不同。

如果你愿意，我下一步可以顺着课件把 TCP 的三个相关量连起来讲清楚：LastByteSent / LastByteAcked / LastByteRcvd 这类图是怎么推 rwnd 的。

来源



回到上面的rtt那块吧，重新给我介绍这块内容

好，我们回到课件里 **TCP 的 RTT 估计与超时重传 (Timeout / Retransmission) ** 这块，按课件思路重新顺一遍。

这块要解决什么问题？

TCP 要可靠，就需要“丢了就重传”。但你怎么判断“丢了”？——靠 **timeout (超时)**。

chapter_3_notes

难点是：网络 RTT 会变，如果 timeout：

- 太小：明明只是延迟，你也会误判成丢包，导致不必要重传
- 太大：真丢包了，你要等很久才重传，恢复慢

所以 TCP 要**动态估计 RTT**，并据此设置超时间隔。

Step 1: 测一个样本 RTT (SampleRTT)

SampleRTT：从某个 segment 发出，到收到它对应 ACK 的时间（一次测量）。

chapter_3_notes

它会抖动很大，所以不能直接拿它当 timeout。

Step 2: 指数平滑得到平均 RTT (EstimatedRTT)

课件用 **指数加权移动平均 (EWMA)**： chapter_3_notes

$$EstimatedRTT \leftarrow (1 - \alpha) \cdot EstimatedRTT + \alpha \cdot SampleRTT$$

常用 $\alpha = 0.125$ 。

直觉：

- 旧值占大头（稳定）
- 新样本占小头（能跟上变化，但不会被一次抖动带飞）

Step 3: 再估计“抖动幅度” (DevRTT)

平均值还不够，因为同样的平均 RTT，有的网络很稳，有的很抖。

课件用：  chapter_3_notes

$$DevRTT \leftarrow (1 - \beta) \cdot DevRTT + \beta \cdot |SampleRTT - EstimatedRTT|$$

常用 $\beta = 0.25$ 。

直觉：DevRTT 越大，说明 RTT 越不稳定。

Step 4: 设置超时 (TimeoutInterval)

课件公式：  chapter_3_notes

$$TimeoutInterval = EstimatedRTT + 4 \cdot DevRTT$$

直觉：

- EstimatedRTT 给“中心位置”
- $4 \cdot DevRTT$ 给“安全余量”：网络越抖，余量越大，避免误判丢包

你刚才那个题的答案（顺便带出来）

“平均 RTT 没变，但网络突然更抖”：主要是 **DevRTT 变大**，从而把 TimeoutInterval 拉大（因为后面有 $+4 \cdot DevRTT$ ）。  chapter_3_notes

10 秒确认题

如果某一段时间网络变得更稳定（SampleRTT 很接近 EstimatedRTT），那么 DevRTT 会趋向变大还是变小？ TimeoutInterval 会怎么变？

来源



变小，趋紧estimatedrtt

对✅：更稳定时 $|SampleRTT - EstimatedRTT|$ 变小，所以 DevRTT 会变小，从而 TimeoutInterval 会更接近 EstimatedRTT（变“紧”）。 [chapter_3_notes](#)

继续课件这一块的下一小节：TCP retransmission 的两个关键坑：重传导致的 RTT 测量歧义 + 解决（Karn/指数退避）。

为什么“重传”会让 SampleRTT 变得不可信？

如果一个 segment 超时后被重传了，然后你收到一个 ACK，你不知道这个 ACK 是：

- 对“第一次发送”的 ACK（只是延迟很久）
- 还是对“重传那次发送”的 ACK

这样测出来的 SampleRTT 就有歧义，会污染 EstimatedRTT。课件通常给的处理原则是：对重传的 segment，不用它来更新 SampleRTT / EstimatedRTT（Karn's algorithm 的核心思想）。 [chapter_3_notes](#)

超时之后怎么调整 timeout？（指数退避 Exponential Backoff）

当发生 timeout（说明网络可能拥塞/很抖），TCP 会把 timeout 变得更保守：

- 下一次的 TimeoutInterval 会增大（常见做法是乘 2），直到收到正常 ACK 再回归。
这就是指数退避的直觉：越不顺，就越别催网络。 [chapter_3_notes](#)

小问题（你答完我继续下一节：TCP 的 ACK 生成策略 / fast retransmit 的铺垫）：

如果发生一次 timeout，你觉得 TCP 把 TimeoutInterval 变大主要是为了避免什么问题？（一句话就行）

来源

📄 🍌 🗨️ ⬆️ ↺️ ...

避免这次timeout是正常情况下的延迟，所以放松一点看能不能就正常收到了

对✅：主要就是为了避免“只是延迟变大/抖动变大”却被频繁误判成丢包，从而反复超时重传、把网络搞得更拥塞。 📄 chapter_3_notes

继续课件：TCP 什么时候发 ACK (ACK generation)，这块是后面“duplicate ACK / fast retransmit”的铺垫。

TCP ACK 生成策略（接收端一般怎么回 ACK）

课件通常总结成几条规则（你可以当做“经验法则”记）：

1. 按序到达的 segment：通常采用“延迟 ACK”（不是每个都立刻回），但会在很短时间内回一个累计 ACK（确认到目前连续收到的最后字节+1）。
2. 乱序到达且中间有缺口：立即发送 duplicate ACK，ACK 仍然指向“缺口处的下一个期望字节”（也就是一直重复同一个 ACK 值）。
3. 收到填补缺口的 segment：如果现在能把连续区间延长，就立刻发送新的累计 ACK。

你只要抓住一个核心：

TCP 的 ACK 是累计的，缺口存在时就会不断重复同一个 ACK 值（duplicate ACK）。

这件事为什么重要？因为发送端看到多个 duplicate ACK，就能推断“不是网络完全断了，而是某个中间段丢了但后面的段还能到”，从而触发 fast retransmit（快速重传）（通常是 3 个重复 ACK 触发）。

小题：帮你把 duplicate ACK 的含义钉死

假设接收端已经按序收到了字节 0..999。此时它期望下一个字节是 1000 (ACK=1000)。如果它先收到了一个乱序 segment，包含字节 1200..1399（中间缺 1000..1199），那么它回的 ACK 应该是多少？为什么？

来源



1000

对✅：还是回 ACK=1000。因为 TCP 是累计 ACK：有缺口时，ACK 始终指向“第一个缺失字节”（next expected byte），即使后面更大的字节段到了，也只会触发 duplicate ACK=1000。

继续课件下一小节：**Fast Retransmit（快速重传）**（不等超时，靠重复 ACK 触发）。

Fast Retransmit：为什么需要它？

仅靠 timeout 的问题是：timeout 往往设得比较保守（防误判），所以丢一个包时等待超时会很慢。

但如果你发现接收方一直在回同一个 ACK（duplicate ACK），说明：

- 后面的数据还能到（链路不是完全断）
- 只是某个“缺口段”没到（很可能丢了）

因此发送端可以**更早地重传缺失段**，加快恢复。

触发规则（课件常见表述）

当发送方收到 **3 个重复 ACK（triple duplicate ACK）**，就认为 ACK 指向的那个段丢了，立即重传（fast retransmit），不等 timer 超时。

你来做个超短判断题

“快速重传”重传的是哪一段？

- A) 最新发送的段
- B) ACK 值所指向的“缺失段”（也就是从 ACK 开始的那段字节）

选一个即可。



pause，课件应该到了tcp reliable data transfer了

已思考 13s >

你说得对，课件这时候确实进入 **3.5 里的“TCP reliable data transfer”** 这一节了：TCP 在不可靠的 IP 之上“造”出可靠传输服务。

TCP reliable data transfer 这一节在讲什么（按课件）

课件先给一个总览：TCP 的 rdt 机制有 3 个关键词：  chapter_3_notes

- **pipelined segments**：不是 stop-and-wait，一次可以在路上飞很多段
- **cumulative ACKs**：ACK 是累计的（确认到某个字节为止）
- **single retransmission timer**：只维护一个重传定时器（看作给“最老的未确认段”用）

以及两种触发重传的方式：

- **timeout events (超时)**
 - **duplicate ACKs (重复 ACK)** (后面引出 fast retransmit)
-

简化版 TCP sender (你一定要抓住 2 个变量 + 1 个 timer)

课件用一个“简化版 sender”来讲 (先忽略 dup ACK、流控、拥塞控)。  chapter_3_notes

两个核心指针

- **SendBase**: 当前“最老的未确认字节”的序号 (base)  chapter_3_notes
- **NextSeqNum**: 下一个要发送的新数据的第一个字节序号  chapter_3_notes

窗口的直觉就是: **[SendBase, NextSeqNum)** 这段字节已经发出但还没被累计确认。

一个定时器 (给谁用?)

- timer 只盯着 **oldest unacked** (也就是 SendBase 对应的那个段/字节范围)
-

三类事件 (这节基本就是这三条)

1) 应用层来数据 (data rcvd from app)

- 生成 segment, seq# = NextSeqNum (注意是**按字节**: segment 里第一个数据字节的编号)
- 交给 IP 发出去
- NextSeqNum 增加 length(data)
- 如果 timer 没在跑: 启动 timer

2) 收到 ACK (ack rcvd)

- 如果 ACK 的值 y **确认了新的数据** ($y > \text{SendBase}$), 就把 **SendBase = y** (累计确认: SendBase-1 是最后一个被累计确认的字节)  chapter_3_notes
- 如果还有未确认段: 开/重开 timer; 否则停 timer

3) 超时 (timeout)

- 重传“最老的未确认段” (最小 seq# 的那段)
 - 重启 timer
-

你确认一下 (一个小题)

假设当前：

- `SendBase = 1000`
- `NextSeqNum = 1600` （说明 1000..1599 都已经发出但未确认）

现在收到一个累计 ACK： $y = 1400$ 。

1. `SendBase` 会变成多少？
2. timer 是更可能“停掉”还是“继续跑”？（提示：是否还有未确认数据）

📄 chapter_3_notes

来源

📄 🍌 🗨️ ⬆️ ↺ ...

1000,继续跑。但是这和gbn有什么区别

你这题里 **SendBase 应该变成 1400**（不是 1000）。因为 `ACK=1400` 的语义是“到 1399 都确认了”，所以最老未确认字节从 1000 推进到 1400。只要还有未确认数据（你这里 `NextSeqNum=1600`，所以 1400..1599 仍未确认），**timer 继续跑**。

那 TCP 这套“看起来像 GBN”，到底和 GBN 有什么区别？

先给你一句最稳的结论：

TCP 的 ACK 语义（累计确认）很像 GBN；但 TCP 不是“纯 GBN”，它在工程上更复杂，尤其是字节流编号、重传触发方式、以及接收端对乱序的处理。

下面按你这章需要的粒度，说清 3 个关键差异（够应付绝大多数考试题）：

1) 编号单位不同：TCP 按“字节”，GBN 按“包”

- **GBN**：seq# 是 packet number (0,1,2...)
- **TCP**：seq# 是 byte number (0,1000,2000... 取决于数据长度)

这导致 ACK 的含义也不同：TCP 的 `ACK=1400` 是“下一期待字节”，而不是“第 1400 个包”。

2) 重传触发更丰富：TCP 除了 timeout，还有 duplicate ACK → fast retransmit

- **GBN**（课件版）：主要靠 timer 超时触发 go-back 重传

- **TCP**: 除了 timeout, 还会利用 **duplicate ACK** 更快推断缺失段并触发重传 (fast retransmit)

这让 TCP 在“只丢了一个中间段、但后面段都到了”的场景下恢复更快。

3) 接收端对乱序段: GBN 明确“不缓存”, TCP 规范允许实现缓存

- **GBN receiver**: 乱序直接丢弃, 只重复 ACK 最后按序的包
- **TCP receiver**: 课件强调“对乱序段怎么处理不强制”, 实现可以选择缓存乱序段; 但 ACK 语义仍是累计到第一个缺口  chapter_3_notes

你确认一下 (只回答一句)

你现在觉得 TCP 更像:

- A) GBN (累计 ACK + 主要确认到缺口前)
- B) SR (逐包 ACK + 必然缓存乱序)

选一个, 并给一句理由 (用“累计 ACK / 逐包 ACK”这种关键词就够)。

来源



gbn, 但是允许缓存乱序字节

对 : TCP 更像 GBN, 因为它的 ACK 语义是 **cumulative** (累计确认到缺口前); 同时实现上可以缓存乱序数据 (但 ACK 仍然不越过缺口)。

继续课件下一节我建议讲 **TCP 的流量控制 (Flow control)**: 它正好把你前面问的 **receive window (rwnd)** 和 “sender 能发多少”串起来。

TCP Flow Control: rwnd 到底怎么限制发送方?

核心目标

避免发送方把接收方缓冲区塞爆: 接收方应用读取速度可能很慢, 所以 TCP 必须“控发送方的在途数据”。

关键变量 (课件通常用这些符号)

接收端维护：

- `RcvBuffer`：接收缓冲区总大小
- `LastByteRcvd`：最后一个已收到字节的序号
- `LastByteRead`：最后一个已被应用从缓冲读走的字节序号

那么接收端可用空间（也就是 `rwnd`）直觉上就是：

$$rwnd = RcvBuffer - (LastByteRcvd - LastByteRead)$$

也就是：总容量减去“还堆在缓冲里没被应用取走的字节数”。

发送端如何用 `rwnd`？

发送端保证：

$$LastByteSent - LastByteAcked \leq rwnd$$

左边就是“我已经发出去但还没被累计确认的数据量”（in-flight 未确认字节）。

接收端会把 `rwnd` 放在 TCP 首部的 `receive window` 字段里持续通告（piggyback 在 ACK 里很常见）。

小题（你答完我继续讲“`rwnd=0` 会怎样/ `persist timer`”）

如果接收端应用读取变慢，导致 `LastByteRead` 增长变慢，那么公式里（`LastByteRcvd - LastByteRead`）会变大还是变小？`rwnd` 会怎样变化？



变小，`rwnd` 变小

结论对：`rwnd` 会变小。但你第一句的方向需要改一下：

- 应用读得慢 $\Rightarrow$ `LastByteRead` 增长慢
- `LastByteRcvd - LastByteRead`（缓冲里“积压未读”的字节数）会 变大
- 所以 `rwnd = RcvBuffer - (...积压...)` 会 变小

继续课件：`rwnd` 变成 0 会发生什么？（以及 `persist timer` 的动机）。

`rwnd = 0`：死锁风险与解决

会发生什么风险？

如果接收方缓冲被占满，通告 $rwnd=0$ ，发送方会停止发送数据（因为不能再塞）。等接收方应用读走一些数据后，接收方应该通告一个新的 $rwnd>0$ 让发送方继续。

风险在于：这个“窗口更新报文”可能丢失。

如果丢了，发送方一直以为 $rwnd=0$ ，就永远不发；接收方在等发送方继续发——潜在死锁。

解决：Persist timer（坚持定时器）

发送方会启动 **persist timer**：周期性地发送一个很小的“window probe”（探测报文）去问：

你现在窗口还是 0 吗？

接收方收到 probe 就会回 ACK（里面带最新 $rwnd$ ），这样即便窗口更新丢了，也能靠 probe 重新同步。

如果你这块 OK，我们下一节就是 **TCP connection management**（三次握手/四次挥手），也是第 3 章最常考的大题之一。



pause，在3次握手前，请回到flow control之前的TCP rdt的sender和sender events那边

已思考 12s >

好，我们回到 **TCP reliable data transfer** 里那张“简化版 TCP sender + sender events”这一块，把它按课件顺一遍（先不碰 flow control / congestion control，也先忽略 duplicate ACK）。

TCP rdt：简化版 sender 的核心（2 个指针 + 1 个定时器）

课件先给 TCP 在 IP 之上“造可靠”的三个关键词：**pipelined segments**、**cumulative ACKs**、**single retransmission timer**。  chapter_3_notes

然后说我们先看“simplified TCP sender”：先忽略 duplicate ACK，也忽略 flow/congestion control。  chapter_3_notes

两个关键变量

- **SendBase**: 当前“最老的、还没被累计确认”的字节序号 (base)。  chapter_3_notes
- **NextSeqNum**: 下一个要发送的新数据段的起始字节序号。  chapter_3_notes

你可以把区间 [**SendBase**, **NextSeqNum**) 理解为: 已经发出但还没被累计 ACK 覆盖的字节范围。

一个定时器 (single retransmission timer)

- 只维护一个 timer, 课件让你把它理解成: 盯着 **oldest unacked segment** (对应 **SendBase**)。  chapter_3_notes

sender 只需要处理三类事件 (课件就是这三条)

事件 A: 上层应用交数据 (data received from application above)

课件伪代码是:

1. 用 **NextSeqNum** 作为 seq# 创建 segment, 然后交给 IP 发出去
2. **NextSeqNum** += length(data) (按字节推进)
3. 如果 timer 现在没在跑 → start_timer

****直觉: ****第一次把“未确认区间”从空变成非空时, 才需要开 timer。

事件 B: 收到 ACK (ACK received with value y)

课件写得很精确: 只在 **y > SendBase** 时才更新:

- **SendBase = y**
- 注释: **SendBase-1** 是“最后一个被累计确认的字节”  chapter_3_notes
- 然后:
 - 如果还有未确认段 → start_timer
 - 否则 → stop_timer  chapter_3_notes

****直觉: ****ACK 是累计的, “一下子确认一大段”很正常; 只要 base 往前推进了, 就要看“未确认集合是否清空”。

事件 C: 超时 (timeout)

课件伪代码:

- 重传 아직未 ACK 的、seq# 最小的那个 segment (也就是最老的那个)
- 然后 start_timer (重启 timer)

你可以把它理解成：TCP 用一个 timer 管“最老欠账”，超时就先把最老那笔补上。

你确认理解的小题（答完我就继续下一页：retransmission scenarios / fast retransmit）

假设此刻：

- $\text{SendBase} = 1000$
- $\text{NextSeqNum} = 1600$ （说明 1000..1599 都已发出但未确认）

现在收到一个 ACK： $y = 1400$ 。

1. SendBase 应该更新成多少？
2. timer 应该停还是继续跑？（理由一句话：是否还有未确认数据）

来源



1400，继续

对 ☒： $\text{SendBase}=1400$ ，而且因为 1400..1599 仍未确认（ $\text{NextSeqNum}=1600$ ），所以 timer 继续跑/重启为盯最老未确认段。

继续课件下一块：retransmission scenarios（重传场景）——也就是“为什么只用一个 timer 也能工作、但会有什么副作用”，以及这如何引出 duplicate ACK / fast retransmit。

TCP 重传典型场景（按课件思路）

场景 1：最老未确认段丢了（经典）

- sender 连续发了很多段（pipelining）
- 中间最老的那个段（对应 SendBase ）丢了
- receiver 因为缺口存在，只能不断回同一个累计 ACK（ACK 不前进）
- sender 如果只靠 timeout：要等 timer 到点才重传最老段

这解释了为什么 timeout 恢复可能慢。

场景 2：ACK 丢了/延迟（会出现“重复 ACK”或“旧 ACK”）

- ACK 乱序/延迟到 sender

- sender 看到 $y \leq \text{SendBase}$ 的 ACK 就当作 duplicate/old ACK (SendBase 不回退)

场景 3：为什么一个 timer 也能行

因为 TCP 的 ACK 是累计的：只要最老缺口补上，ACK 会一下子跳很远，把后面已经到达/缓存的部分一并确认（实现允许缓存乱序）。

所以“先救最老欠账”是有道理的。

这就引出：duplicate ACK 能提供更快的“丢包信号”

当 receiver 收到缺口后面的段时，会不断发同一个 ACK (duplicate ACK)。sender 看到多个 duplicate ACK，就有理由不等 timeout，直接重传缺口段 (fast retransmit)。

📄 chapter_3_notes

小题（帮你把“重复 ACK 代表什么”说清）

如果 sender 连续收到很多个 **ACK=1400**（值完全一样），最合理的解释是：

A) receiver 在说“我很喜欢 1400”

B) receiver 在说“我一直缺从 1400 开始的那段字节（缺口还没补上），所以我的累计 ACK 不前进”

来源

📄 🍌 🗨️ ⬆️ ↺️ ...

b

对 ☒：重复收到同一个 **ACK=1400** 的含义就是“缺口还在：我一直在等从 1400 开始的那段字节，所以累计 ACK 不前进”。

继续课件同一页往下：**Fast Retransmit（快速重传）**——TCP 里利用 duplicate ACK 比 timeout 更快地触发重传。 📄 chapter_3_notes

Fast Retransmit：规则与直觉

- 触发条件：发送方收到 **3 个重复 ACK (triple duplicate ACK)**
- 动作：立即重传 **ACK 所指向的缺失数据段**（也就是从该 ACK 值开始的那段字节），不等 timer 超时。 📄 chapter_3_notes

直觉：

- 如果后面的段还能到达 receiver，说明网络没完全断
- receiver 却反复说 "ACK=1400" ⇒ 大概率是 "1400.." 这段丢了
- 这比 timeout 更早、更可靠地提示"该补洞了"

10 秒确认题

在 fast retransmit 里，重传的是哪一段？

A) 最新发送的那段 (NextSeqNum 附近)

B) 从 **重复 ACK 的值** 开始的那段 (例如重复 ACK=1400 ⇒ 重传起始 seq=1400 的段)

选一个即可。

来源



b

对 ☒：重传的是 **从重复 ACK 的值开始的那段** (重复 ACK=1400 ⇒ 优先重传起始序号为 1400 的段)，而不是“最新发的那段”。  chapter_3_notes

到这里，TCP 的 rdt (sender 事件 + timeout + dupACK/fast retransmit) 这块就基本完整了。下一节课件通常就是 **TCP connection management (三次握手)**。

你要我现在继续讲 **3-way handshake** 吗？

来源



嗯

已思考 17s >

好，我们进入课件 **Connection Management → TCP 3-way handshake** 这一节。它的目标很简单：在真正传数据前，双方先“握手”，**确认彼此都愿意建立连接**，并且协商连接参数（尤其是双方各自的初始序号 ISN/seq# 等）。

TCP 3 次握手（按课件图逐步走）

课件把 client/server 的状态画在一起：server 先在 **LISTEN**，client 主动发起连接。

第 1 次：Client → Server: SYN

- 报文：SYNbit=1, Seq=x
- 动作：client 选择一个初始序号 x，发 SYN 请求建立连接
- client 状态：SYN_SENT

第 2 次：Server → Client: SYN + ACK (SYNACK)

- 报文：SYNbit=1, Seq=y, ACKbit=1, ACKnum=x+1
- 动作：server 也选择自己的初始序号 y，并用 ACKnum=x+1 确认收到了 client 的 SYN (SYN 占 1 个序号)
- server 状态：SYN_RCVD
- 课件还强调：server 在这里会 **create new socket for communication back to client** (也就是 accept 后为该连接创建“真正通信的 socket”)

第 3 次：Client → Server: ACK (可捎带数据)

- 报文：ACKbit=1, ACKnum=y+1 (并且这段可能携带 client→server 的数据)
- 关键语义 (课件原话):
 - client 收到 SYNACK 表示 **server is live**
 - server 收到最后这个 ACK 表示 **client is live**
- 最终双方进入 ESTAB (连接建立完成)

你可以用一句话记住“为什么要 3 次”

2 次只能做到“我想连/我也想连”；第 3 次是为了让 server **确认 client 确实收到了 server 的 SYN (也就是 y)**，从而双方对“彼此的初始序号”达成一致，并且互相确认对方在线 (live)。

10 秒小题 (你答完我再继续下一页：握手的 FSM 图)

假设 client 选 x=100，server 选 y=300：

1. server 发回 SYNACK 的 ACKnum 是多少？

2. client 最后发回 ACK 的 ACKnum 是多少？



101, 301

对✅：SYNACK 的 ACKnum=101，最后 client 的 ACKnum=301（因为 SYN 也占用 1 个序号）。

继续课件紧跟着的一页：3-way handshake 的状态机（FSM）怎么读，以及一个经典面试/考试点：为什么 server 需要 SYN_RCVD 这个半开状态。

3-way handshake 的 FSM 你怎么读（抓住状态 + 触发事件）

Client 侧（主动打开 active open）

- CLOSED →（主动 connect，发送 SYN）→ SYN_SENT
- SYN_SENT →（收到 SYN+ACK，发送 ACK）→ ESTABLISHED

Server 侧（被动打开 passive open）

- CLOSED →（listen）→ LISTEN
- LISTEN →（收到 SYN，回 SYN+ACK）→ SYN_RCVD
- SYN_RCVD →（收到最后 ACK）→ ESTABLISHED

为什么 server 要有 SYN_RCVD（半开连接）？

因为 server 在回了 SYN+ACK 之后，还没收到 client 的最后 ACK，此时它不能认为连接已经真正建立完成，只能暂时保留一些连接状态（比如对方地址/端口、初始序号 y、期望的 ACK=y+1 等）等待确认。

这也会自然引出“半开连接被滥用”的安全话题（SYN flood），但如果课件没展开，你记住：SYN_RCVD 就是“我回复了，但还没最终确认”的等待区。

你确认一下（1 句话）

server 在 SYN_RCVD 等的那件事是什么？（等哪个报文、满足什么条件就转到 ESTABLISHED）



client对server的回复

对✅：server 在 SYN_RCVD 等的是 client 发回的最后那个 ACK（确认 ACKnum = y+1），收到后才进入 ESTABLISHED。

接下来课件通常就是 TCP connection closing（四次挥手）。我按同样方式只讲这一节，讲完给你小题。

TCP 连接关闭：四次挥手（FIN）

关闭的关键词：FIN 表示“我这边没数据要发了”。因为 TCP 是全双工，两边的发送方向是独立的，所以关闭也常常需要“四步”才能让两边都各自关掉自己的发送方向。

第 1 次：Client → Server：FIN

- client 发 FIN（并带 seq=u）表示：我发完了
- client 进入 FIN_WAIT_1

第 2 次：Server → Client：ACK

- server 回 ACK (ack=u+1) 确认收到了 FIN
- server 进入 CLOSE_WAIT（表示“对方不发了，但我这边可能还要发数据”）
- client 进入 FIN_WAIT_2

第 3 次：Server → Client：FIN

- 等 server 也发完了，它再发自己的 FIN (seq=v)
- server 进入 LAST_ACK

第 4 次：Client → Server：ACK（并进入 TIME_WAIT）

- client 回 ACK (ack=v+1) 确认 server 的 FIN
- server 收到后进入 CLOSED
- client 会进入 TIME_WAIT 等一小段时间再 CLOSED（确保最后 ACK 丢了还能重发、避免旧报文干扰新连接）

小题（确认你理解全双工）

为什么 server 在收到 client 的 FIN 之后，不一定立刻发 FIN，而是可能先回 ACK 进入 CLOSE_WAIT？（一句话）



可能server还有东西没发完

对✅：因为 TCP 是全双工，client 发 FIN 只表示“client→server 方向不再发了”，但 **server→client 方向可能还有数据要发**，所以 server 先 ACK 并进入 CLOSE_WAIT，等自己也发完了再发 FIN。

第 3 章到这里主线就基本走完了（传输层服务→复用分用→UDP→可靠传输原理→TCP rdt→流控→连接管理）。如果你愿意，我可以接着用 10 分钟把“最容易考的综合题型”帮你串成一个答题模板：

- 给一个抓包/时间线，填 SYN/ACK/FIN 的 seq/ack；
- 给一个丢包场景，判断会出现 duplicate ACK / timeout / fast retransmit；
- 给一个 rwnd 变化，判断 sender 能不能继续发。



这个是个很好的提议，但是请你在帮我介绍题型之前，把剩下的内容走完，还剩principles of congestion control等等等等

已思考 16s >

好，我们继续走完后面的内容。先从课件 **3.6 Principles of Congestion Control（拥塞控制原理）** 这一节开始（只讲这一节，不把 3.7 TCP congestion control 一口气全倒出来）。

3.6 拥塞是什么？和 flow control 有啥本质区别？

课件定义的拥塞很“口语”：太多源以太快的速率发送，网络（尤其路由器队列）扛不住。它的典型表现是：

- 路由器 buffer 溢出 → 丢包
- 排队越来越长 → 时延很大

关键点：拥塞控制 ≠ 流量控制。

- flow control (rwnd) 是“别把接收方缓冲塞爆”（端到端：发端→收端）

- congestion control 是“别把网络塞爆”（全局：很多流共享瓶颈链路）

课件用 3 个 scenario 讲“拥塞的代价”到底是什么

Scenario 1: 无限 buffer、没有重传

- 两个发送方共享一个输出链路容量 R
- 吞吐上限是每条连接最多 $R/2$
- 但当到达速率 λ_{in} 逼近容量时，排队时延会爆炸式变大（即使不丢包）

直觉：buffer 无限并不“解决问题”，它只是把“丢包”换成“排队很久”。

Scenario 2: 有限 buffer + timeout 重传（会更糟）

一旦 buffer 有限就会丢包，而传输层看到 timeout 就会重传。结果是：

- 路由器实际看到的输入不再只是“原始数据 λ_{in} ”，还叠加了“重传数据 λ'_{in} ”
- 这会让网络更拥塞，形成恶性循环

课件还强调一个很容易被忽略的“成本”：

- **无谓重传**：其实包没丢，只是 ACK/时延抖动导致 sender 误超时，于是同一个包被网络搬运了两份，吞吐反而下降

Scenario 3: 多跳、多流共享瓶颈（“上游带宽白费”）

当一个包最后在某个路由器因为拥塞被丢掉时：

- 这个包在“上游链路上已经占用的传输容量”全都白费了
- 拥塞越严重，这种浪费越多

这一节你需要记住的 2 句话（考试型）

1. 拥塞的两大外显症状：丢包 + 时延（来自路由器队列）  chapter_3_notes
2. 拥塞的隐性成本：无谓重传 + 上游容量浪费

小问题（你答完我就进入下一页：两种拥塞控制方法 + TCP 选哪种）

如果路由器 buffer “无限大”，你觉得网络在高负载下更可能出现的是：

- A) 大量丢包为主
- B) 丢包不多，但排队时延巨大为主

选一个并说一句原因即可。

[来源](#)

b

对 ☒：buffer 无限时更可能是 B：丢包不多，但排队时延巨大，因为包不容易被溢出丢掉，而是堆在队列里等。

继续课件：3.6 里下一页通常讲“拥塞控制的两种大类：end-to-end vs network-assisted”，以及 Internet/TCP 采用哪一种。

拥塞控制两大路线

1) End-to-end congestion control（端到端）

网络层/路由器不给明确反馈；发送方通过端系统观测来推断拥塞：

- 丢包（timeout 或重复 ACK）
- RTT/排队时延的上升

然后根据推断调整发送速率。

TCP 属于这一类（经典 Internet 的设计哲学）。

2) Network-assisted congestion control（网络辅助）

路由器显式给发送方反馈“拥塞了/快拥塞了”：

- 直接告诉允许的发送速率
- 或者用标记/bit（比如显式拥塞通知 ECN 这种思路）

TCP 端到端拥塞控制的“宏观目标”

课件通常会强调三个目标词：

- **Efficiency（高利用率）**：尽量用满瓶颈带宽
 - **Fairness（公平）**：多条连接共享瓶颈时别一家独大
 - **Low delay（低时延）**：别把队列撑满导致巨延迟（这一点有时是隐含目标）
-

10 秒小题（为下一节 TCP congestion control 铺路）

TCP 在端到端拥塞控制里，最常用的“拥塞信号”是什么？

- A) rwnd 变小
- B) 丢包/重复 ACK/超时（推断网络在拥塞）



b

对 ☒：TCP 主要把 丢包/重复 ACK/超时当作拥塞信号（端到端推断）。

下面进入课件 3.7 TCP Congestion Control（TCP 拥塞控制）。我先讲最核心的“cwnd/发送速率怎么被控制 + Slow Start / Congestion Avoidance / AIMD”的主线；其他细节（比如 fast recovery）我放下一段。

3.7 TCP Congestion Control：发送方到底怎么“限速”？

1) 关键变量：cwnd（congestion window）

发送方维护一个 cwnd，表示“为了不让网络拥塞，我最多允许自己在路上的未确认数据量（字节）”。

所以发送方实际能同时在途的数据受两者共同限制：

$$\text{in-flight} \leq \min(\text{cwnd}, \text{rwnd})$$

- rwnd：别撑爆接收方（flow control）
- cwnd：别撑爆网络（congestion control）

2) “速率”怎么近似？

课件常用近似：

$$\text{TCP sending rate} \approx \frac{\text{cwnd}}{\text{RTT}}$$

直觉：一轮 RTT 内，你大概能发 cwnd 这么多并收到 ACK，然后再发下一轮。

TCP 如何调整 cwnd：AIMD 思想

课件主线：

- 网络“看起来没拥塞” → cwnd 慢慢涨（探测更高带宽）
- 检测到拥塞（丢包/超时/3 dup ACK） → cwnd 立刻降（让出负载）

这就是经典 **AIMD: Additive Increase, Multiplicative Decrease**：

- 加性增：线性加一点点
- 乘性减：一下砍一大截

两个阶段：Slow Start + Congestion Avoidance

课件把 cwnd 的增长分成两种“速度”：

(1) Slow Start（慢启动，其实是指数涨）

名字叫“慢”，但增长很快：每收到一个 ACK，就让 cwnd 增加 1 MSS，于是每 RTT cwnd 近似翻倍（指数增长）。

(2) Congestion Avoidance（拥塞避免，线性涨）

当 cwnd 超过某个阈值 **ssthresh** 后，进入拥塞避免：每 RTT 线性增加大约 1 MSS（加性增）。

拥塞事件发生时怎么调（先讲最常考的骨架）

课件通常分两种信号强弱：

- **timeout（强拥塞信号）**：通常更激进地降 cwnd（回到较小值，重新 slow start）
- **triple duplicate ACK（较弱拥塞信号）**：一般砍半 cwnd，并进入快速恢复/拥塞避免（细节下一段讲）

你先回答一个确认题（我用来决定下一段讲 fast recovery 的细节）

Slow Start 里为什么说 cwnd 是“每 RTT 近似翻倍”？（用“每个 ACK +1 MSS”的链式理由回答一句就行）



1mss是什么意思

MSS 是 Maximum Segment Size: TCP 一个 segment 里最多能装多少 **应用数据 (payload)** 的字节数 (不含 TCP/IP 头)。

所以看到 “+ 1 MSS” 的意思是：

- cwnd (拥塞窗口) 是按“字节”计的
- 每次把 cwnd 增加 一个段的数据量大小 (大约就是一次能多放进网络里的“一个 segment 的 payload 大小”)



继续

好，继续刚才那题：**Slow Start 为什么每 RTT 近似翻倍**，以及接着把 **ssthresh**、**超时/3dupACK 时 cwnd 怎么变**讲完主线。

Slow Start：为什么“每 RTT 近似翻倍”

Slow Start 的规则 (课件表述)：**每收到一个 ACK，就让 cwnd 增加 1 MSS。**

直觉链条是：

- 在某个 RTT 开始时，cwnd 允许你在途大概有 k 个 MSS 的数据
- 这些数据会带来大约 k 个 ACK (每个 ACK 确认一段新数据)
- 每个 ACK 都让 cwnd +1 MSS
- 所以一个 RTT 结束时，cwnd 大约从 k 变成 $2k$ (翻倍)

ssthresh：从“指数探测”切到“线性探测”

课件用 **ssthresh (slow start threshold)** 作为分界：

- 当 $cwnd < ssthresh$ ：Slow Start (指数涨)
- 当 $cwnd \geq ssthresh$ ：Congestion Avoidance (线性涨：每 RTT 约 +1 MSS)

拥塞事件发生时：cwnd 和 ssthresh 怎么更新 (骨架版)

课件一般这样讲 (细节不同版本 TCP 可能稍有差异，但课内主线是这个)：

情况 A: timeout (强拥塞信号)

- $ssthresh = cwnd/2$ (先把阈值设成“当前的一半”)
- $cwnd$ 降到很小 (常见是 1 MSS), 然后重新 Slow Start

情况 B: 3 duplicate ACK (较弱拥塞信号)

- $ssthresh = cwnd/2$
- $cwnd$ 通常降到大约一半, 然后进入“快速恢复/拥塞避免” (下一段讲 fast recovery 的细节)

你先做个小判断 (确认你掌握“信号强弱”)

为什么课件把 **timeout** 看成比 **3 duplicate ACK** 更“强”的拥塞信号? (一句话直觉: 哪个说明网络更糟?)



timeout更糟更挤, ack都回不来了; 我还是没理解cwnd是什么意思

你第一句理解对: **timeout** 往往意味着更严重 (连 ACK 都等不到, 可能丢得多/队列更爆), 而 3 dup ACK 说明“后面的包还能到, 网络没彻底糊”。

下面把 **cwnd** 讲到你能“用来做题”的程度。

cwnd 到底是什么? (一句话 + 一个画面)

****cwnd (congestion window) ****就是发送方自己设的一个“额度/上限”:

为了不把网络挤爆, 我最多允许自己同时有这么多 ****未被 ACK 的数据 (字节) ****在网
络里飞着。

把它想成“高速路的限行”: 你最多允许自己有 $cwnd$ 这么多车在路上, 超过就先别放车进去。

cwnd 怎么真正“卡住发送”?

发送方始终维护两样东西 (和你前面 TCP rdt 那块完全能对上):

- **LastByteSent**: 我已经发出去的最后一个字节序号

- `LastByteAcked` : 我已经被累计确认的最后一个字节序号

那么当前在途未确认的数据量就是：

$$\text{in-flight bytes} = \text{LastByteSent} - \text{LastByteAcked}$$

拥塞控制要求它不能超过 `cwnd`（同时也不能超过 `rwnd`）：

$$\text{LastByteSent} - \text{LastByteAcked} \leq \min(\text{cwnd}, \text{rwnd})$$

这句话的意思就是：

- 如果你已经有 `cwnd` 字节在路上了，你就必须等 ACK 回来（让 `LastByteAcked` 前进），才能继续发新的字节。

一个具体数值例子（你立刻就能感觉 `cwnd` 在干嘛）

假设：

- `cwnd = 10,000 bytes`
- `rwnd` 很大（不限制）
- 你每个 segment payload 约 `MSS=1000 bytes`

那发送方大概能让 **10 个 segment** 同时在路上。

如果 ACK 还没回来，你第 11 个就先发不出去——这就是 `cwnd` 的“限速”。

`cwnd` 为什么会变化？

因为 TCP 用它来“探测并适应网络可承受的负载”：

- 没看到拥塞信号 → `cwnd` 增大（你就能放更多数据在路上，吞吐变高）
- 看到拥塞信号（丢包/dupACK/timeout） → `cwnd` 减小（你就少放点，缓解拥塞）

10 秒确认题（非常关键）

如果此刻：

- `LastByteSent - LastByteAcked = 8000 bytes`
- `cwnd = 9000 bytes`
- `rwnd` 很大

问：你现在还能再发送多少字节的新数据而不违反 `cwnd`？（算一下就行。）



对✅：还能再发 **1000 bytes**，因为 cwnd 允许在途最多 9000，你现在在途 8000，还剩 1000 的额度。

继续课件下一块：**AIMD 的图像直觉 + 公平性 (Fairness)**，以及最常考的结论：两条 TCP 流共享瓶颈时为什么会“趋向公平”。

AIMD：为什么它会趋向公平？

课件常用一个“二维图”来讲：横轴是 flow1 的速率，纵轴是 flow2 的速率。

- **容量约束线**： $x_1 + x_2 = R$ （总速率不能超过瓶颈容量）
- **公平线**： $x_1 = x_2$ （两者平分）

AIMD 的动作：

- **Additive Increase (加性增)**：两条流都以相似步幅增加 → 在图上朝右上方走（沿着 45° 方向靠近容量线）
- **Multiplicative Decrease (乘性减)**：拥塞时都按比例砍（比如砍半）→ 在图上朝原点方向缩（保持比例关系，但离容量线远一点）

反复“线性涨 + 成比例砍”，几何上会把点推向 **公平线与容量线的交点附近**，所以长期看会趋向公平。

一个很常考的一句话

TCP 用 AIMD 在效率和公平之间做折中：

- 增的时候慢慢探测（避免一下冲爆）
- 看到拥塞就大幅退让（让出资源）

小题（你答完我继续讲 fast recovery 以及 “cwnd vs ssthresh 的典型曲线”）

如果两条 TCP 流共享同一个瓶颈链路，并且它们都按 AIMD 调整 cwnd，那么在长期平均意义上，它们会更倾向于：

- A) 一条占满，另一条饿死
- B) 分到比较接近的带宽

选一个并说一句理由（用“加性增/乘性减”关键词）。



b原理模糊；而且cwnd每次rtt+1和cwnd每次rtt double是两种策略嘛

选 B 是对的。你问的两个点我一起捋清楚：(1) 公平性为什么成立，(2) cwnd 线性涨 vs 翻倍涨是不是两种策略。

1) 公平性直觉：为什么 AIMD 会把两条流拉向“差不多”？

把两条流的速率想成 x_1, x_2 。

- **加性增 (Additive Increase)**：两条流每轮都“加同样的一点点”（大概每 RTT +1 MSS）。
这意味着：谁小谁相对增长更快（小的加 1 占比更大），所以差距会被慢慢缩小。
- **乘性减 (Multiplicative Decrease)**：拥塞时两条流都“按比例砍”（比如都砍半）。
按比例砍不会让强者更强，它只是把大家一起拉下来，下一轮又继续“同样+1”的追赶。

所以反复下去，会趋向让两条流在瓶颈处共享得更接近（这就是课件图里往公平线靠的直觉）。

2) cwnd “每 RTT +1” vs “每 RTT 翻倍”是两种策略吗？

是的，而且它们对应 TCP 拥塞控制的两个阶段：

(a) Slow Start：指数增长（每 RTT 近似翻倍）

规则是“每收到一个 ACK，cwnd + 1 MSS”，一轮 RTT 会收到很多 ACK，于是 cwnd 一轮下来大约翻倍。

用途：刚开始不知道带宽多大，先快速探测。

(b) Congestion Avoidance：线性增长（每 RTT 大约 +1 MSS）

当 cwnd 到达阈值 ssthresh 以后，改成线性增长。

用途：接近瓶颈时别冲太猛，稳一点探测。

一句话：翻倍涨是“快速起步找区间”，+1 是“稳态下小步试探”。

10 秒确认题

如果当前 cwnd 还很小（远低于网络可承受范围），你觉得用 Slow Start（翻倍涨）更合理，还是直接 Congestion Avoidance（线性+1）更合理？为什么一句话。



所以指数增长类似刚冷开始的时候，之后可能经过一些次的乘性砍之后就线性调整？

对，整体节奏你说得基本就是课件主线 ，我帮你把“什么时候指数、什么时候线性、什么时候乘性砍”用最标准的方式钉死：

- **指数增长 (Slow Start)**：通常发生在
 1. 连接刚开始 (cold start)
 2. 或者发生 **timeout** 这类“强拥塞信号”之后，cwnd 被降到很小（常见 1 MSS），需要重新探测这时每 RTT 近似翻倍快速把 cwnd 拉起来。
- **乘性砍 (Multiplicative Decrease)**：当检测到拥塞（丢包/dupACK/timeout）时触发常见是把 **ssthresh** 设为当前 cwnd 的一半，并把 cwnd 大幅降低（timeout 更狠；3 dup ACK 通常减半）。
- **线性调整 (Congestion Avoidance)**：当 cwnd 达到/超过 **ssthresh** 后进入这时每 RTT 大约 +1 MSS，小步试探，直到下一次拥塞信号再乘性砍。

所以你可以把一条 TCP 的 cwnd 轨迹想成“锯齿”：

- 开头可能指数上升到某个阈值
- 然后线性上升
- 一旦拥塞就砍半（或更狠）
- 再线性上升……（timeout 才会回到很小再指数爬）

如果你 OK，我下一段就把课件里 3 dup ACK vs timeout 的具体 cwnd/ssthresh 更新 + fast recovery（如果课件包含 Reno 那部分）讲完，然后第 3 章就真的收尾了。



ssthresh是怎么维护的量

****ssthresh (slow start threshold) ****是 TCP 发送方维护的一个“阈值”，用来决定 **cwnd** 该用指数涨 (Slow Start) 还是线性涨 (Congestion Avoidance)。

它怎么维护 (课件主线)

ssthresh 不是固定常数，而是在发生拥塞事件时更新的：

- 一旦检测到拥塞 (典型是 丢包/timeout 或 3 dup ACK)，就把

$$ssthresh \leftarrow \frac{cwnd}{2}$$

(也就是把“新的阈值”设成当时拥塞前窗口的一半)。

然后接下来：

- 如果 cwnd 被降得很小 (常见 timeout 情况)，你会在 **cwnd < ssthresh** 区间走一段 Slow Start (指数涨)，涨到接近 ssthresh 后再转线性。
- 如果只是 3 dup ACK 这种较弱信号，cwnd 往往直接降到一半附近，那么通常立刻就在阈值附近做线性增长 (具体细节取决于 Reno/fast recovery 那页)。

直觉

ssthresh 像一个“记忆”：

我上次把网络挤爆时，大概是在 cwnd≈这个量级，所以以后别用指数冲过头；到这个附近改成线性更稳。

如果你愿意，我可以用一条完整时间线举例 (cwnd 初始 1 MSS → 指数涨 → 到 ssthresh 转线性 → 3 dup ACK 砍半并更新 ssthresh → 继续线性...) 把它串成一眼能做题的流程。



好的

已思考 20s >

好，我们用一条“时间线例子”把 **cwnd / ssthresh / SS / CA / Reno 快速恢复**串起来，你做题就会很稳。下面严格按课件规则来：

- Slow Start**：初始 cwnd=1 MSS，每 RTT 近似翻倍 (实现上每个 ACK 让 cwnd +1 MSS)。  chapter_3_notes
- Congestion Avoidance**：每 RTT 线性增加约 1 MSS (实现上每个 ACK 增加 1 MSS/cwnd)。  chapter_3_notes

- **ssthresh**: 检测到丢包后设为 $cwnd/2$; $cwnd < ssthresh$ 用 SS, $cwnd \geq ssthresh$ 用 CA。  chapter_3_notes
 - **Reno 快速恢复**: 3 个重复 ACK 检测丢包时: $ssthresh = cwnd/2$, $cwnd = ssthresh + 3$, 之后线性增长。  chapter_3_notes
 - **timeout**: $ssthresh = cwnd/2$, $cwnd = 1 \text{ MSS}$, 重传丢失段。  chapter_3_notes
-

一条完整例子

0) 初始 (冷启动)

假设初始 $ssthresh = 16 \text{ MSS}$ (阈值一开始可以是系统给的某个值, 我们只关心后续怎么更新它)。

$cwnd = 1 \text{ MSS} \rightarrow$ 因为 $cwnd < ssthresh$, 走 **Slow Start**。  chapter_3_notes

1) Slow Start: 指数涨到阈值附近

按“每 RTT 翻倍”的直觉:  chapter_3_notes

- RTT1 结束: $cwnd \approx 2$
- RTT2: $cwnd \approx 4$
- RTT3: $cwnd \approx 8$
- RTT4: $cwnd \approx 16$

到这里 $cwnd \geq ssthresh$, 切换到 **Congestion Avoidance** (线性涨)。

 chapter_3_notes

2) Congestion Avoidance: 线性涨

接下来每 RTT 大约 +1 MSS:  chapter_3_notes

- RTT5: $cwnd \approx 17$
- RTT6: $cwnd \approx 18$
- RTT7: $cwnd \approx 19$
- RTT8: $cwnd \approx 20$

3) 发生丢包: 3 个 duplicate ACK (Reno 的“较温和”反应)

假设在 $cwnd = 20$ 时触发 3 dup ACK:

- $ssthresh = cwnd/2 = 10$
- $cwnd = ssthresh + 3 = 13$ (进入快速恢复 FR, 并线性增长)  chapter_3_notes

直觉: 没 timeout, 说明网络还“能动”, 所以 Reno 不会把 $cwnd$ 打回 1, 而是在半幅附近继续。  chapter_3_notes

4) 又过了一段，发生 timeout（更强拥塞信号）

假设后来 cwnd 又长到 18，结果 timeout：

- $ssthresh = 18/2 = 9$
- $cwnd = 1 \text{ MSS}$ （回到 Slow Start 起点）  chapter_3_notes

接下来因为 $cwnd < ssthresh (=9)$ ，先 SS：1→2→4→8→（下个翻倍会到 16，但到达/超过阈值就切 CA），于是到 9 附近就改回线性 CA。  chapter_3_notes

你做题时怎么用这条线？

看到题目给你“触发条件”（timeout or 3 dup ACK）+ “当前 cwnd”，你就能立刻写出：

1. $ssthresh = cwnd/2$
2. cwnd 会掉到哪里（timeout → 1；3dupACK/Reno → $ssthresh+3$ ）
3. 下一步走 SS 还是 CA（比较 cwnd 和 ssthresh）

小题（你答完我继续下一节：TCP throughput 那两页）

在上面例子里，3 dup ACK 时 $cwnd=20$ ：

1. ssthresh 是多少？
2. Reno 进入 FR 后 cwnd 立刻变成多少？

来源



$ssthresh+3$ ；所以是单纯tahoe和reno下timeout的时候都会到1，reno下fr下是sshthresh吗

你问得很准，我把 **Tahoe vs Reno** 的“cwnd 掉到哪儿”用最清晰的方式对齐一下（按课件常见版本）：

先把你这题补完

当 3 dup ACK 发生且当时 $cwnd=20$ 时：

- $ssthresh = 20/2 = 10$

- **Reno** 进入 fast recovery 的“立即设置”是： $\text{cwnd} = \text{ssthresh} + 3 = 13$ （不是单纯 ssthresh ）  chapter_3_notes

Tahoe vs Reno：区别主要在“3 dup ACK 怎么处理”

两者在 **timeout** 上通常是一致的（强拥塞信号）：

- $\text{ssthresh} = \text{cwnd}/2$
- $\text{cwnd} = 1 \text{ MSS}$
- 然后走 slow start  chapter_3_notes

差别在 **3 dup ACK**：

- **Tahoe**：把 3 dup ACK 也当成很严重 → 也会 $\text{cwnd}=1 \text{ MSS}$ ，重新 slow start（更“保守”）。
- **Reno**：认为 3 dup ACK 说明网络还在跑（只是丢了一个段） →
 - $\text{ssthresh}=\text{cwnd}/2$
 - $\text{cwnd}=\text{ssthresh}+3$ 进入 fast recovery
 - 收到对“重传段”的新 ACK 后， cwnd 回到 ssthresh ，进入 congestion avoidance（线性涨）。  chapter_3_notes

所以你最后那句要这样记更准确：

- **Reno 在 fast recovery 期间不是直接 $\text{cwnd}=\text{ssthresh}$ ，而是先 $\text{ssthresh}+3$** （因为已经看到 3 个 dup ACK，意味着有 3 段数据离开网络、可以“临时”把窗口撑起来一点）；等新 ACK 到来后再回到 ssthresh 进入 CA。  chapter_3_notes

如果你 OK，下一节课件一般会讲 **TCP throughput（吞吐近似公式）**：把“锯齿形 cwnd ”换算成平均吞吐，这也是常考计算题。

来源



继续

已思考 18s >

好，我们继续课件里 **TCP throughput (平均吞吐)** 这一小节：它的核心是在 **AIMD** 的“锯齿形 cwnd”上，算一个“长期平均速率”。

这页在干嘛：把 cwnd 的锯齿，换算成 avg throughput

课件先明确了前提：

- 忽略 slow start (只看稳态的 CA/FR 那种锯齿)
- 永远有数据要发 (sender 不会因为没数据而停)

关键定义：W 是什么？

- W = 发生丢包时的窗口大小 (以 bytes 计)

在 AIMD 里，一旦丢包就会把窗口 砍半，所以丢包后窗口从 W 变成 $W/2$ ，然后又线性加回去到 W ，再丢，再砍半……形成锯齿。

平均窗口为什么是 $3/4 W$ ？

因为锯齿在一个周期内大致在 $[W/2, W]$ 之间线性变化，所以平均值就是区间平均：

$$avg\ cwnd = \frac{W + W/2}{2} = \frac{3}{4} W$$

课件直接给结论：avg window size (in-flight bytes) = $3/4 W$ 。

平均吞吐怎么来？

“吞吐 $\approx$ 每 RTT 能成功发/确认的数据量”，而稳态下每 RTT 大概能送出 $\approx cwnd$ 字节，所以：

$$avg\ throughput \approx \frac{avg\ cwnd}{RTT} = \frac{3/4 W}{RTT} \text{ bytes/sec}$$

10 秒小题 (你答完我再继续下一页：TCP fairness 那两页)

如果某连接在丢包前的窗口 $W = 64\text{KB}$ ， $RTT = 100\text{ms}$ ，按课件这个近似公式，平均吞吐大约是多少 **KB/s**？ (提示： $(3/4)W/RTT$)

📄 🍷 🗨️ ⬆️ ↺️ …

480?

已思考 21s >

对的，就是 **480 KB/s**: $(3/4) \cdot 64\text{KB} = 48\text{KB}$ ，再除以 0.1s 得 480KB/s 。

 chapter_3_notes

继续课件下一节：**TCP Fairness (公平性)**。

TCP Fairness: 课件想让你记住什么

公平性的“目标句”

如果 **K 条 TCP 连接** 共享同一个瓶颈链路 (带宽 R)，理想情况是每条平均速率约 R/K 。

为什么 TCP (AIMD) 会趋向公平?

课件给的直觉就是两句话：

- **additive increase**: 两条连接在拥塞避免阶段都“每 RTT 加一点” (图上表现为 $\text{slope} \approx 1$ 的往右上走)
- **multiplicative decrease**: 一旦丢包，两条连接都按比例减 (例如窗口减半)，所以“占得多的那条也会按比例退让更多”

反复“线性加 + 比例砍”，就会把两条连接拉向“比较均分”的区域 (课件图里是往 **equal bandwidth share** 靠)。

但公平性也有两个常考 caveat

1. 多媒体/实时流常用 UDP，不想被 TCP 的拥塞控制限速，所以它们可能“不公平地占更多”。  chapter_3_notes
2. 应用可以开 **多个并行 TCP 连接** (浏览器常见)：开的连接数越多，拿到的总带宽越多 (课件给了“1 条 vs 11 条”的例子)。

小题 (你答完我就把第 3 章最后一页 summary 走完)

同一条瓶颈链路带宽 R ，已有 9 条 TCP 连接在用。

新应用如果开 **1 条 TCP**，大概拿到多少？如果开 **11 条 TCP**，又大概拿到多少？ (用 $R/_$ 的形式写即可)

来源

     ...

第一问对 ：开 1 条时总共 10 条连接，所以大约拿 $R/10$ 。

第二问要改一下：如果新应用开 11 条，那总连接数变成 $9 + 11 = 20$ 。

- 每条连接平均约 $R/20$
- 但新应用有 11 条连接，所以它拿到的总带宽约：

$$11 \cdot (R/20) = 11R/20$$

而不是 $R/20$ 。

到这里 **congestion control** 主线也走完了。课件第 3 章剩下通常就是一个 **chapter summary**（把 UDP/TCP、rdt、GBN/SR、connection management、congestion/flow control 的关键词串起来）。你要我把 summary 也按“只讲一节”的方式过一遍吗？

     ...

还有一个 network-assisted congestion control

已思考 15s >

对，这章里还有一小节是 **network-assisted congestion control**（网络辅助拥塞控制）：意思是路由器不再“沉默”，而是主动给端系统反馈拥塞信息。  chapter_3_notes

Network-assisted：路由器能怎么“帮忙”？

课件给了两类典型做法：  chapter_3_notes

1. **1-bit 拥塞信号**：路由器用一个 bit 告诉端系统“我这里拥塞了”（典型例子是 **ECN**）。
2. **显式速率（explicit rate）**：路由器直接告诉发送方“你最多按多少速率发”。

这和 end-to-end（TCP 传统方式）对比就是：

- end-to-end：端系统只能靠 **loss/delay** 自己猜拥塞
- network-assisted：网络（路由器）**显式告诉你“拥塞了/该减速了”**  chapter_3_notes

课件重点：ECN（Explicit Congestion Notification）

ECN 是对 IP 和 TCP 的扩展，让网络能显式向 TCP 发送方/接收方发出拥塞信号。

📄 chapter_3_notes

你可以把 ECN 的直觉理解成一句话：

路由器在快要拥塞时，先“标记”而不是“丢包”，让 TCP 提前减速。

然后 TCP 端收到这种“拥塞信号”后，就会像看到丢包一样，去降低发送速率（本质就是调小 cwnd），只不过信号来源从“丢包推断”变成了“路由器明确提示”。

你来确认一下（选一个）

network-assisted 相比 end-to-end 的最大区别更接近下面哪个？

- A) 还是靠丢包/超时来推断
- B) 路由器会给端系统显式反馈（比如 ECN 的标记 bit 或显式速率）

来源

📄 🍌 🗨️ ⬆️ ↺️ ...

b

对✅：关键区别就是 B：路由器显式反馈（标记 bit / 显式速率），而不是端系统纯靠丢包/时延去猜。

到这里，第 3 章的主干内容（rdt→GBN/SR→TCP rdt→flow control→connection mgmt→congestion control→fairness→network-assisted/ECN）就都走完了。

你要不要我按你之前说的方式，开始整理“常考题型模板”了？

📄 🍌 🗨️ ⬆️ ↺️ ...

要

好，我按“最常考的综合题型”给你一套可直接套用的答题模板（每类都给：看什么 → 写什么 → 常见坑）。我先出这一节；你看完如果想加练，我再给你配套练习题。

题型 1：3 次握手 / 4 次挥手 填 seq/ack（最常考）

你看题时先写两条规则

- $ACKnum = \text{对方 Seq} + \text{该报文携带的数据字节数}$
- SYN/FIN 各占 1 个序号（即使不带数据）

3-way handshake 模板

1. C→S: SYN, Seq=x
2. S→C: SYN+ACK, Seq=y, ACK=x+1
3. C→S: ACK, ACK=y+1（可捎带数据）

4-way close 模板

1. A→B: FIN, Seq=u
2. B→A: ACK, ACK=u+1
3. B→A: FIN, Seq=v
4. A→B: ACK, ACK=v+1（A 进入 TIME_WAIT）

常见坑：把 FIN/SYN 当成 0 字节不占序号（错）。

题型 2：给你一段“乱序/丢包时间线”，判断会出现 duplicate ACK / fast retransmit / timeout

你先记住 TCP ACK 的一句话

ACK 永远指向“第一个缺失字节”（累计确认）；缺口存在时会重复同一个 ACK 值（duplicate ACK）。

快速判断法

- 收到很多重复 ACK=K：说明接收方一直缺从 K 开始的那段
 - 3 个重复 ACK：发送方触发 fast retransmit，重传“从 K 开始的段” [📄 chapter_3_notes](#)
 - timeout：通常表示更严重（ACK 都回不来），会触发超时重传 + 拥塞窗口更狠下降（后面题型 5）
-

题型 3：GBN vs SR 行为判断（尤其是“中间缺一个，后面到了”）

一句话区分

- **GBN receiver 不缓存乱序**：乱序就丢，重复 ACK 最后按序的包；sender 超时会“从缺口开始全重传”。
- **SR receiver 缓存乱序 + 逐包 ACK**：缺口补上后能把缓存连续交付；sender 只重传超时的那一个包。

常见坑：把“ACK 乱序到达”误当成“receiver 能越过缺口 ACK 更大值”（GBN 不可能；TCP 的 ACK 也不越过缺口）。

题型 4：流量控制 rwnd 计算 + sender 能不能继续发

必背公式（接收方视角）

$$rwnd = RcvBuffer - (LastByteRcvd - LastByteRead)$$

应用读慢 $\Rightarrow$ (LastByteRcvd - LastByteRead) 变大 $\Rightarrow$ rwnd 变小。 [chapter_3_notes](#)

sender 约束（概念版）

$$in\text{-}flight = LastByteSent - LastByteAcked \leq rwnd$$

所以还能发的字节数 = rwnd - in-flight （若为负则不能发）。 [chapter_3_notes](#)

常见坑：把 rwnd 和 cwnd 混在一起（rwnd 管接收方 buffer，cwnd 管网络拥塞）。

题型 5：拥塞控制 cwnd/ssthresh 更新（Tahoe/Reno）+ 画 cwnd 曲线

先写“二阶段增长”

- $cwnd < ssthresh$ ：Slow Start（每 RTT 近似翻倍） [chapter_3_notes](#)
- $cwnd \geq ssthresh$ ：Congestion Avoidance（每 RTT 约 +1 MSS） [chapter_3_notes](#)

遇到拥塞事件怎么写（最常考）

- **timeout**：ssthresh = cwnd/2，cwnd = 1 MSS（回 SS） [chapter_3_notes](#)
- **3 dup ACK (Reno)**：ssthresh = cwnd/2，cwnd = ssthresh + 3（fast recovery），新 ACK 后回到 ssthresh 进入 CA [chapter_3_notes](#)
- **3 dup ACK (Tahoe)**：更保守，通常也会 cwnd=1 重新 SS（如果课件强调 Tahoe vs Reno 的差异，就按它写）

题型 6：吞吐近似（锯齿平均）

稳态锯齿：窗口在 $[W/2, W]$ 线性变化

- 平均窗口 $\approx 3W/4$
- 平均吞吐 $\approx (3/4)W/RTT$  chapter_3_notes

你选一个方向，我们下一步就做“真题级练习”

你想先刷哪一类？

1. seq/ack 填空（握手/挥手）
2. dup ACK / fast retransmit 时间线推理
3. Reno/Tahoe cwnd 曲线与更新计算

来源



FSM应该怎么看；或者应该怎么画。我觉得我最大的问题就是确定不了一个过程有几个状态

已思考 19s >

你这个感觉非常正常：“一个过程到底要几个状态”不是你不会，而是取决于你画 FSM 的抽象层级。网络课里的 FSM 其实大多是 **extended FSM**（扩展有限状态机）：真正的“状态”除了圈圈里的名字，还包含了变量（seq#、expected、base/next、timer on/off、buffer 里有什么）。变量一多，如果硬把每种取值都画成一个圈，状态会爆炸。

下面给你一套我自己画/读课件 FSM 的固定方法，你照着做基本不会迷路。

1) 先确定你画的是“纯 FSM”还是“扩展 FSM”

- **Stop-and-wait (rdt2.2/rdt3.0)**：变量很少（seq 只有 0/1 + 是否在等 ACK），可以画成少量离散状态圈。
- **窗口协议 (GBN/SR/TCP sender)**：有 base/next、window、buffer、timer...如果你硬画圈，状态数几乎无限。正确做法是：画 2~3 个大状态 + 用变量写在状态里（课件很多也是这么干的）。

所以你“确定不了有几个状态”，通常出现在第二类——不是你问题，是模型本来就应该“变量化”。

2) 画 FSM 的万能套路：状态 = “我现在在等什么 + 我手里握着什么关键信息”

具体做法：

Step A：列出事件（events）（一般就这三类）

1. 上层调用： `rdt_send(data)` / 应用给数据
2. 下层到包： `rdt_rcv(pkt)` / 收到 segment 或 ACK
3. 超时： `timeout`

Step B：列出关键变量（state variables）

只保留“会导致行为不同”的变量。典型：

- Stop-and-wait: `seq` (0/1)、是否“已发送未确认”、`lastpkt`
- GBN: `base`、`nextseqnum`、`timer`、`send buffer`
- SR: 在 GBN 基础上再加 per-pkt 的 ACK 记录 + `receiver buffer`

Step C：分状态的原则（核心）

如果在某个变量取值下，对同一个事件的反应（动作/转移）不一样，那就必须分成不同状态。

反过来：如果反应完全一样，就可以合并成一个状态，用变量区分即可。

你可以把“状态圈”理解成：一组“行为规则完全相同”的情形。

3) 用一个你最熟的例子：为什么 rdt3.0 sender 是 4 个状态？

关键变量只有两个维度：

- `seq` $\in \{0, 1\}$
- 我是在“等上层数据”还是“等这个 `seq` 的 ACK”（以及 `timer` 是否开）

所以自然是 $2 \times 2 = 4$ 个状态：

- `WaitCall0`：等上层给数据，发 `seq=0`
- `WaitAck0`：已经发了 `seq=0`，在等 `ACK0`（可能 `timeout` 重传）
- `WaitCall1`
- `WaitAck1`

你看，这不是背出来的，是按“行为是否不同”自动分出来的：

WaitCall0 收到上层数据要“发包+开timer”； WaitAck0 收到上层数据要“忽略/缓存”（行为不同）→ 必须分状态。

receiver 更简单：只需要记 $expected \in \{0,1\} \rightarrow 2$ 个状态。

4) 为什么到 GBN/SR/TCP sender 你会崩：状态其实“画不完”

以 GBN sender 为例，如果你硬要把每个 $(base, nextseqnum)$ 组合都画成一个圈，那状态数随序号空间增长，等价于无限。

所以正确姿势是：

4.1 画“大状态”（通常 2 个就够）

- “可以发送新数据（窗口未满）”
- “窗口满了只能等 ACK”

4.2 把“细节”放到圈里当变量/条件

在转移边上写：

- 事件：收到 ACK(y)
- 条件： $y > base$
- 动作： $base = y + 1$ ，如果 $base == nextseqnum$ stop timer else restart timer
- 事件： timeout
- 动作：重传从 base 开始的未确认段

这就是课里常见的 **FSM + 变量伪代码**混合表达。

你可以把它理解成：圈圈不再枚举所有情况，只表示“当前遵循哪套规则”；具体进度由变量表示。

5) 你怎么“读课件 FSM”不迷路：只盯两件事

每看到一个状态圈，问自己两句：

1. 我现在在等什么事件？（data? ACK? FIN? SYN? timeout?）
2. 发生事件后我会做什么关键动作？（发什么包、更新哪个变量、开/关哪个 timer）

把每条边按固定格式读：

事件/条件 → 动作（发包/改变量/启动停止timer） → 下一个状态

6) 你练“确定状态数”的小练习（我建议你当场做）

不用看课件也能做：

题：Stop-and-wait + seq=0/1 + 需要处理 timeout 重传

问 sender 最少几个状态？

你按我给的维度拆：

- seq 两种
 - 等上层 vs 等 ACK 两种
- 4 个状态（就是 rdt3.0 的结果）

如果你愿意，我们下一步就用你课件里你最卡的那张 FSM（比如 GBN sender 或 TCP 连接管理），我带你按这套流程“从事件+变量推导出状态”，你会发现状态数其实是你自己推出来的，而不是猜的。你把那一页截图发我就行。



4. Consider the reliable data transfer 3.0 protocol. If the network connection between the sender and receiver can **reorder** messages (that is, that two messages propagating in the medium between the sender and the receiver can be reordered), then will the alternating-bit protocol (that is, only using 0 and 1 to denote the sequence numbers of messages) work correctly? If not, draw a diagram showing that. Your diagram should have the sender on the left and the receiver on the right, with the time axis running down the page, showing data and acknowledgement message exchange. Don't forget the sequence number associated.

这个题

已思考 1m 12s >

这题的关键是：rdt3.0/alternating-bit（只用 0/1 序号）默认网络不会让“很久以前的旧分组”在你序号回绕以后又冒出来。一旦允许 reordering + 任意延迟，就会出现“旧包被当成新包”的歧义，所以不再保证正确。

下面给一个标准反例（会导致 重复交付 + 丢失新数据）：

反例时间线（Sender 左，Receiver 右，时间向下）

记三段应用数据分别是 A、B、C。序号只用 0/1。

yaml

```

Sender (S)                                     Receiver (R)
time ↓

S: send  pkt(seq=0, data=A)  ----->    (这个包被网络长时间延迟,

(timeout)

S: retransmit pkt(seq=0, data=A)  ----->    R: expected=0, 收到s
                                           R: deliver A
                                           R: send ACK0  <-----

S: recv ACK0, send pkt(seq=1, data=B)  ----->    R: expected=1, 收到s
                                           R: deliver B
                                           R: send ACK1  <-----

S: recv ACK1, send pkt(seq=0, data=C)  ----->    (C 在路上)

(此时, 最开始那个“很久以前”的 A 终于到了, 并且发生了重排序)
old pkt(seq=0, data=A)  ----->    R: expected=0 (因为
                                           R: 误以为这是“新一轮”的
                                           R: deliver A (A 被重
                                           R: send ACK0  <-----

pkt(seq=0, data=C) arrives  ----->    R: 此时 expected 已
                                           R: 把 C 当成重复/旧的 ;
                                           R: send ACK0  <-----

S: 收到 ACK0, 会以为 C  已经被确认 (因为它现在等的就是ACK0)
但实际: C  没交付, A  被交付了两次。

```

[复制代码](#)

结论：协议失败（A 重复交付、C 丢失）。

失败原因就是：序号空间只有 2 (0/1)，一旦旧包在序号回绕后才到达，接收方无法区分“旧的 seq0”还是“新的 seq0”。

这题想让你写的“一句话结论”

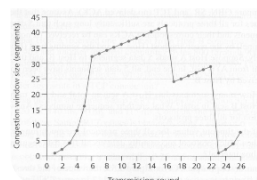
- 不正确（在允许 reordering/任意延迟的网络中）。
- 需要更大的序号空间 + “分组寿命上界”（现实里 TCP 通过大序号 + MSL/TIME_WAIT 等机制避免这种歧义）。

如果你愿意，我也可以顺便告诉你：如果题目里的“reorder”只指短时间乱序、且网络保证“旧包不会活到下一轮序号回绕”，那 ABP 仍然可以工作；但一般这题默认的是可能出现旧

包延迟导致回绕歧义，所以答案是上面的反例。



ving Figure. Assuming TCP Reno is the protocol exper
uestions. In all cases, you should provide a short dis



is the 70th segment ser



已思考 43s >

选 A) 7。

理由（按“每个 transmission round 最多发 cwnd 个 segment”的常见题意）：

- 从图上读出慢启动阶段的窗口大约是：
Round 1~6 的 cwnd 分别约为 1, 2, 4, 8, 16, 32（指数翻倍）。
- 所以前 6 轮一共能发的 segment 数是：
 $1 + 2 + 4 + 8 + 16 + 32 = 63$ 。
- 第 70 个 segment 需要再发 $70 - 63 = 7$ 个，而第 7 轮的 cwnd 已经大于等于 7（图上 round 7 的窗口约 33），所以第 70 个 segment 会在第 7 轮发送出去。

